two-level page table lookup

two-level page table lookup

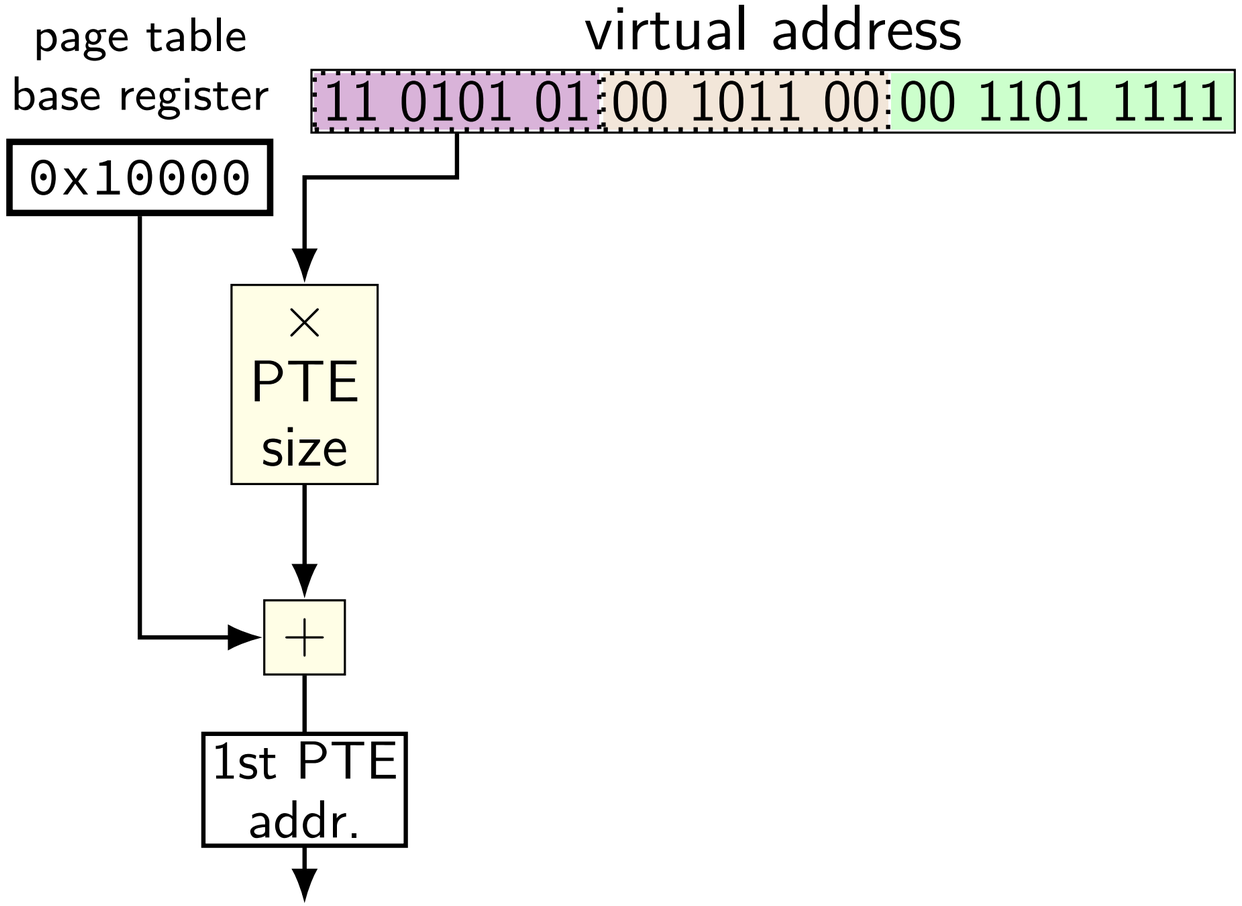
virtual address



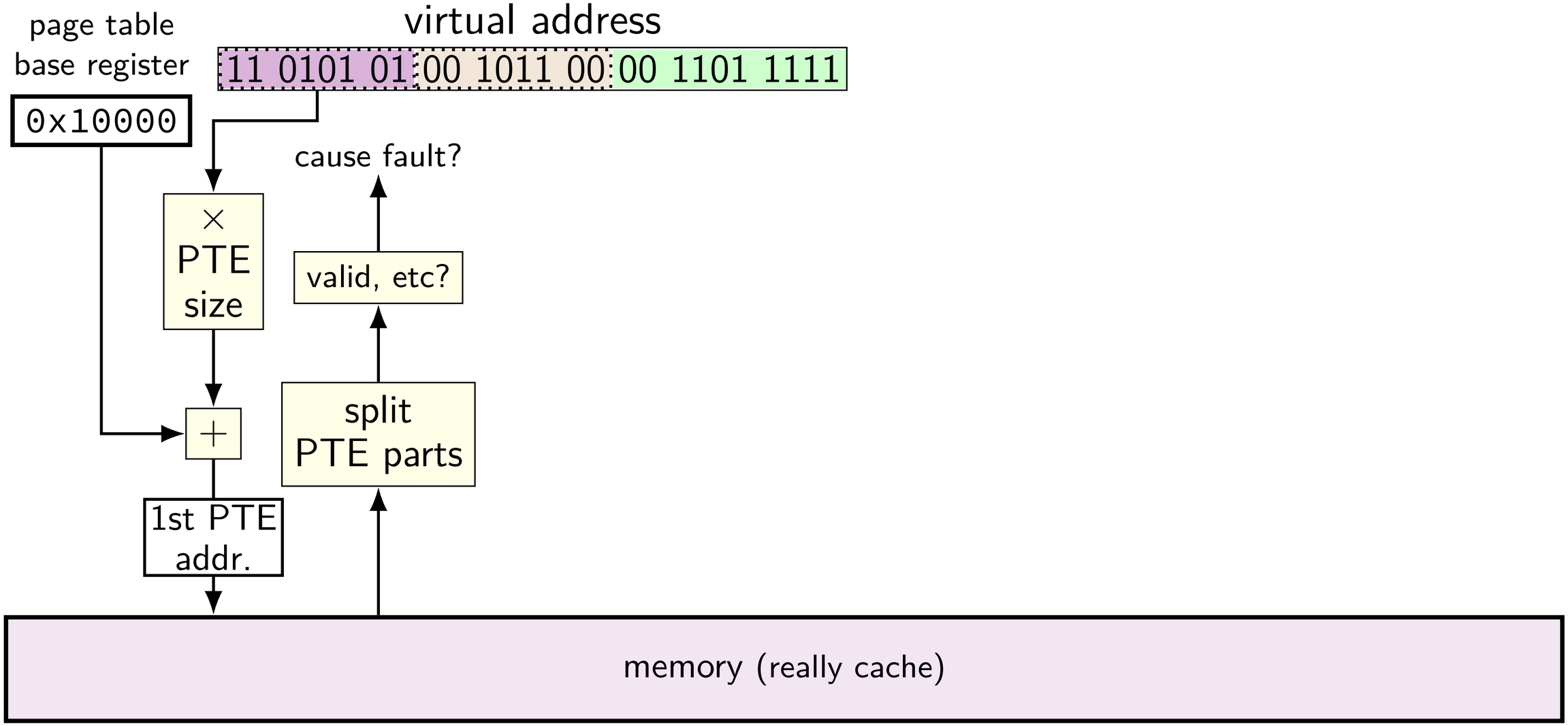
VPN — split into two parts (one per level)

this example: parts equal sized — common, but not required

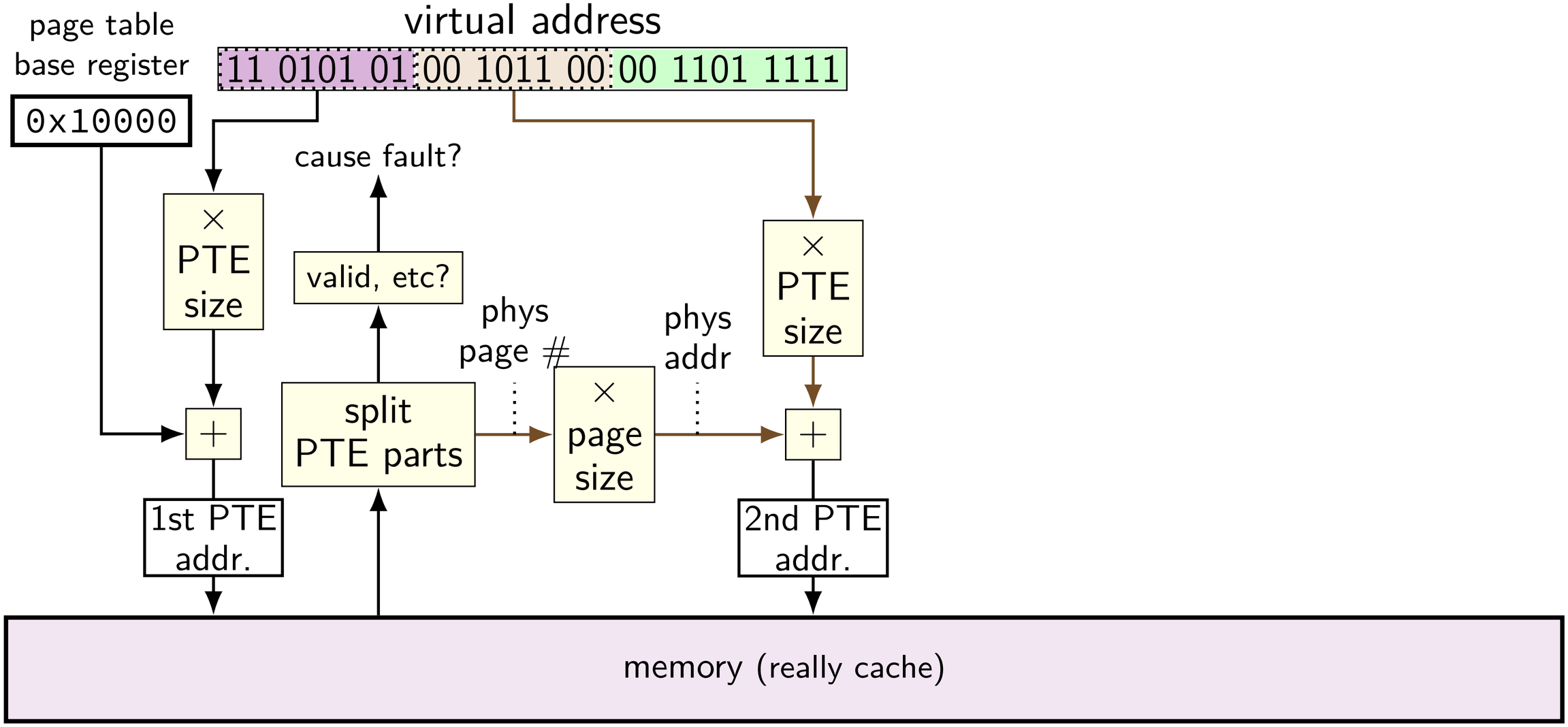
two-level page table lookup



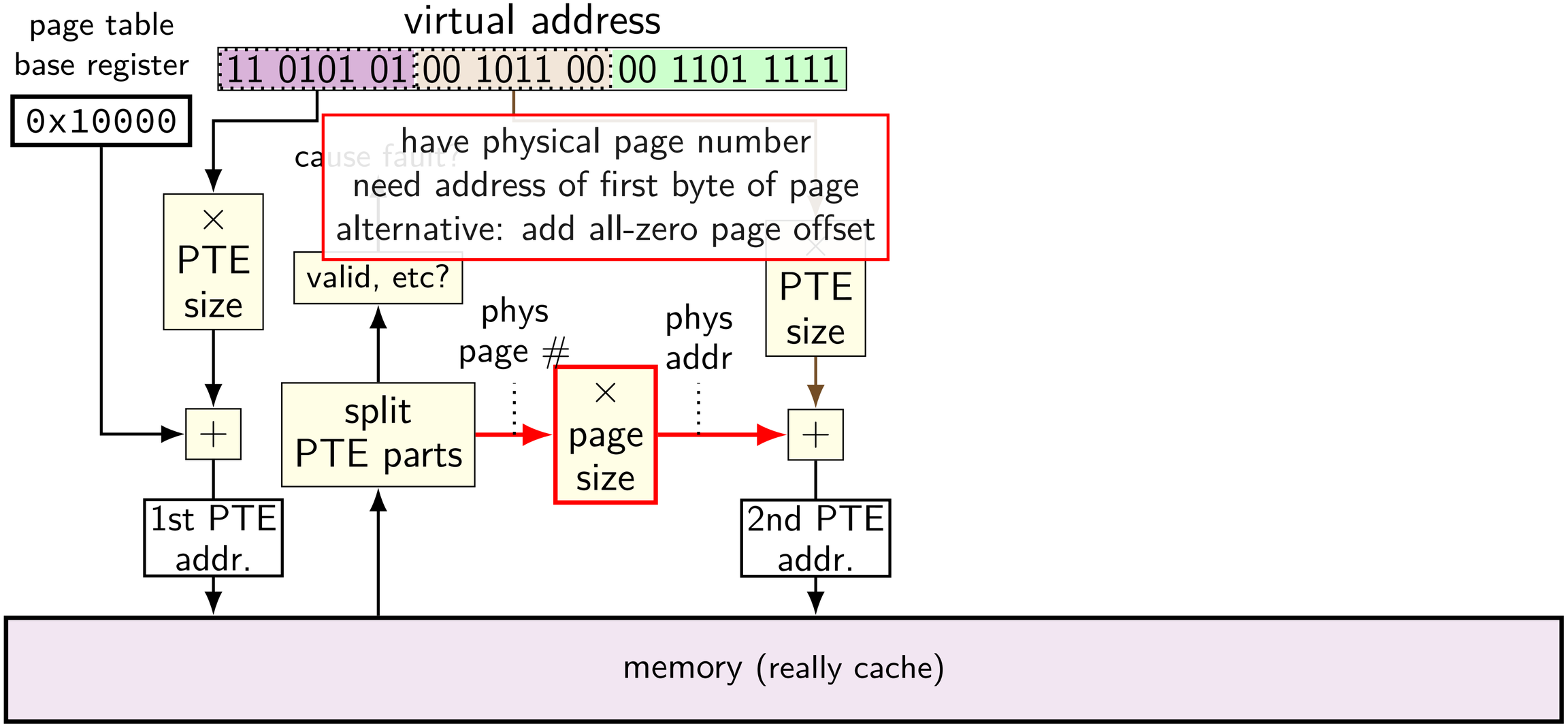
two-level page table lookup



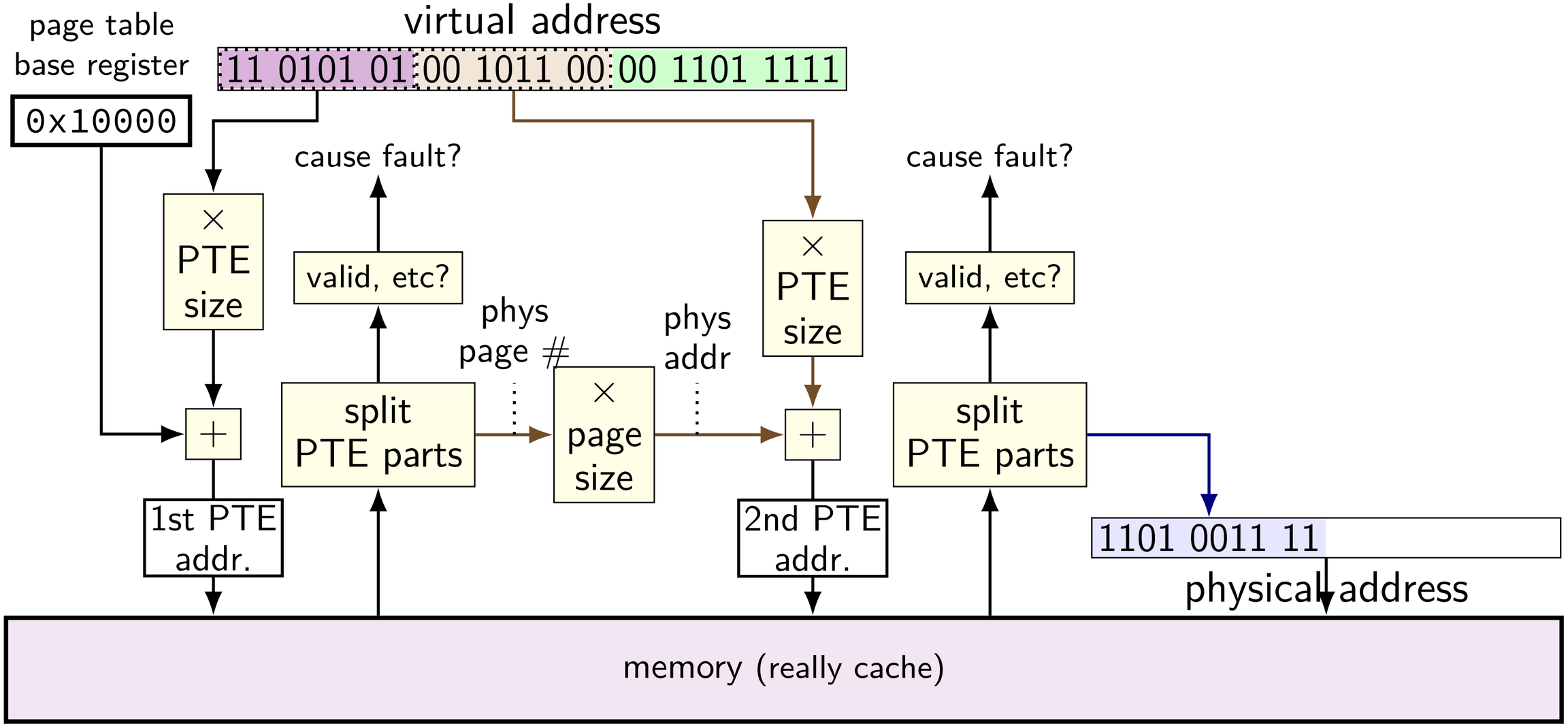
two-level page table lookup



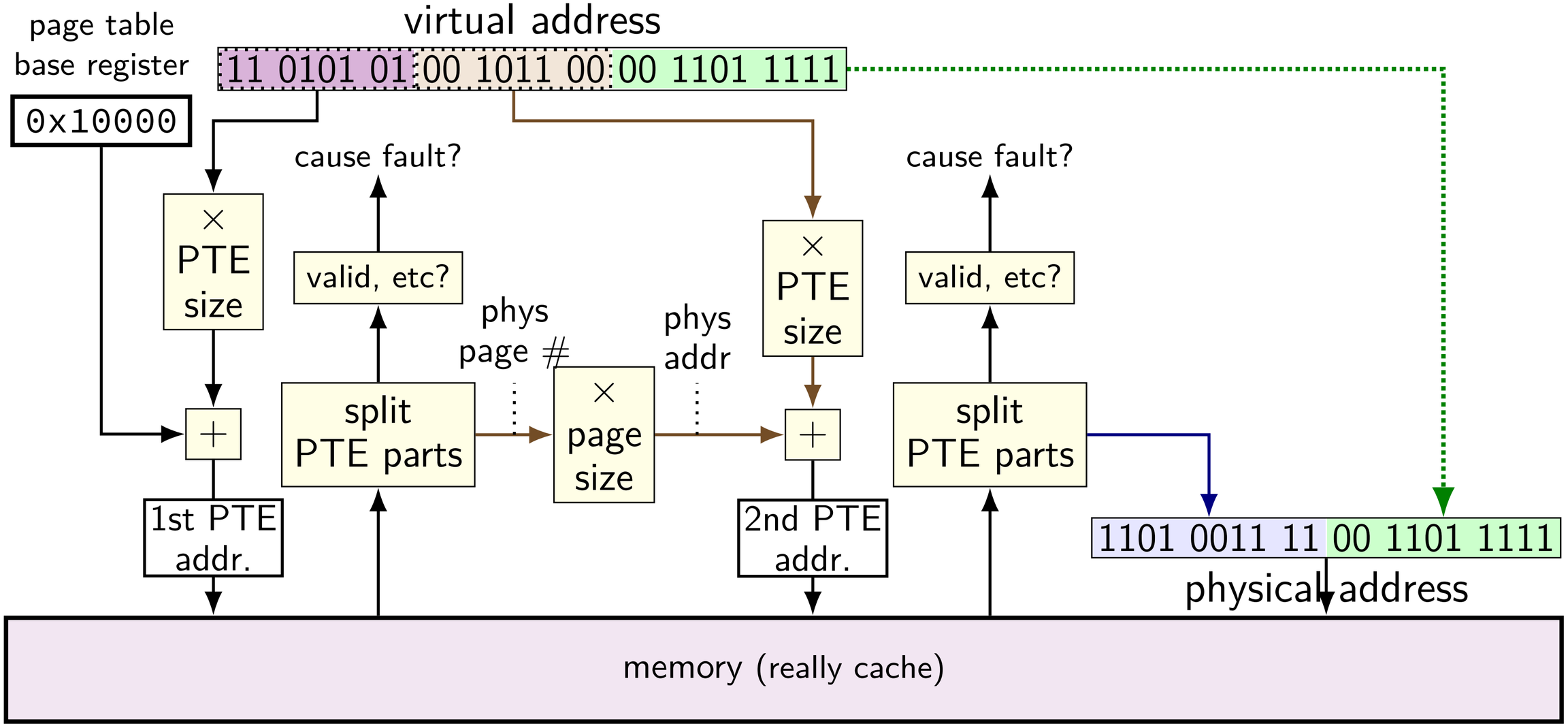
two-level page table lookup



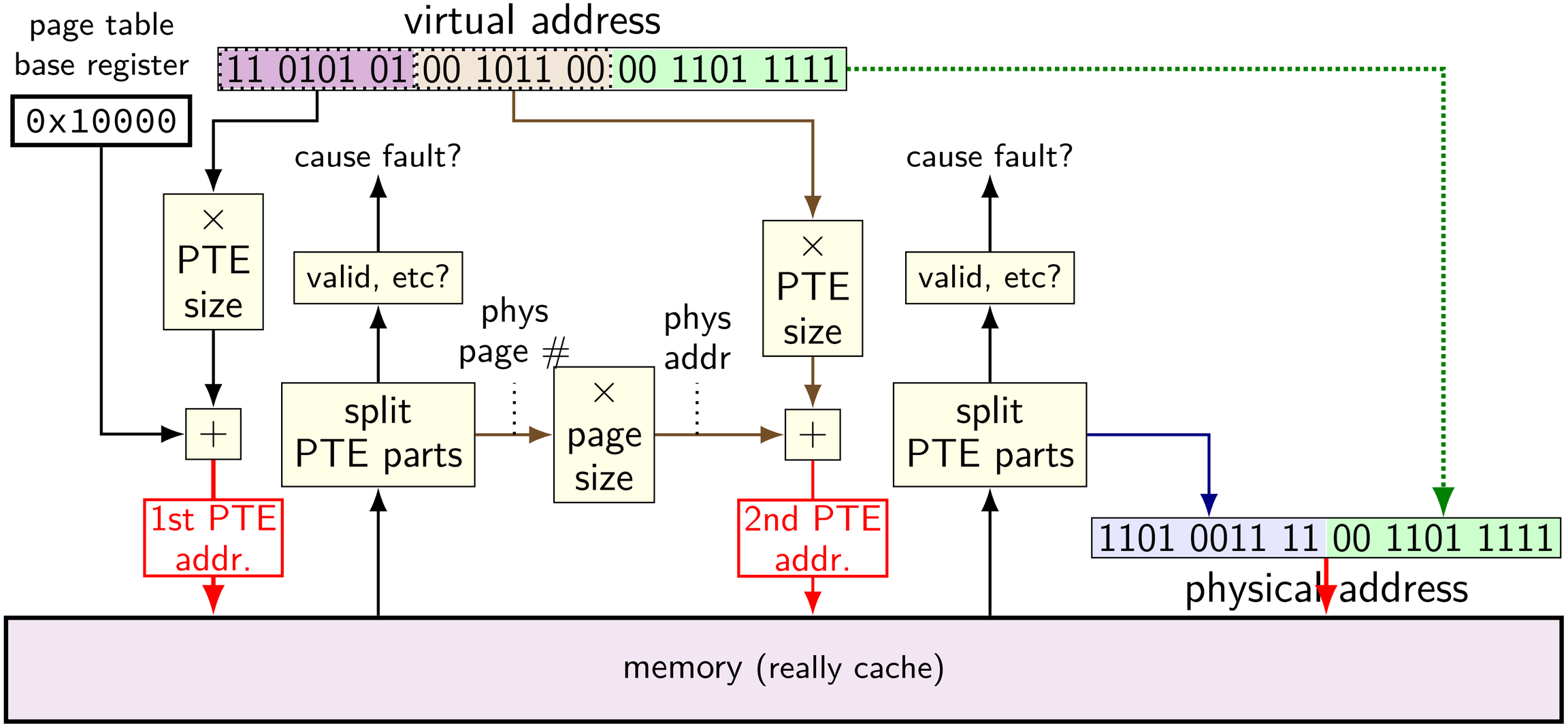
two-level page table lookup



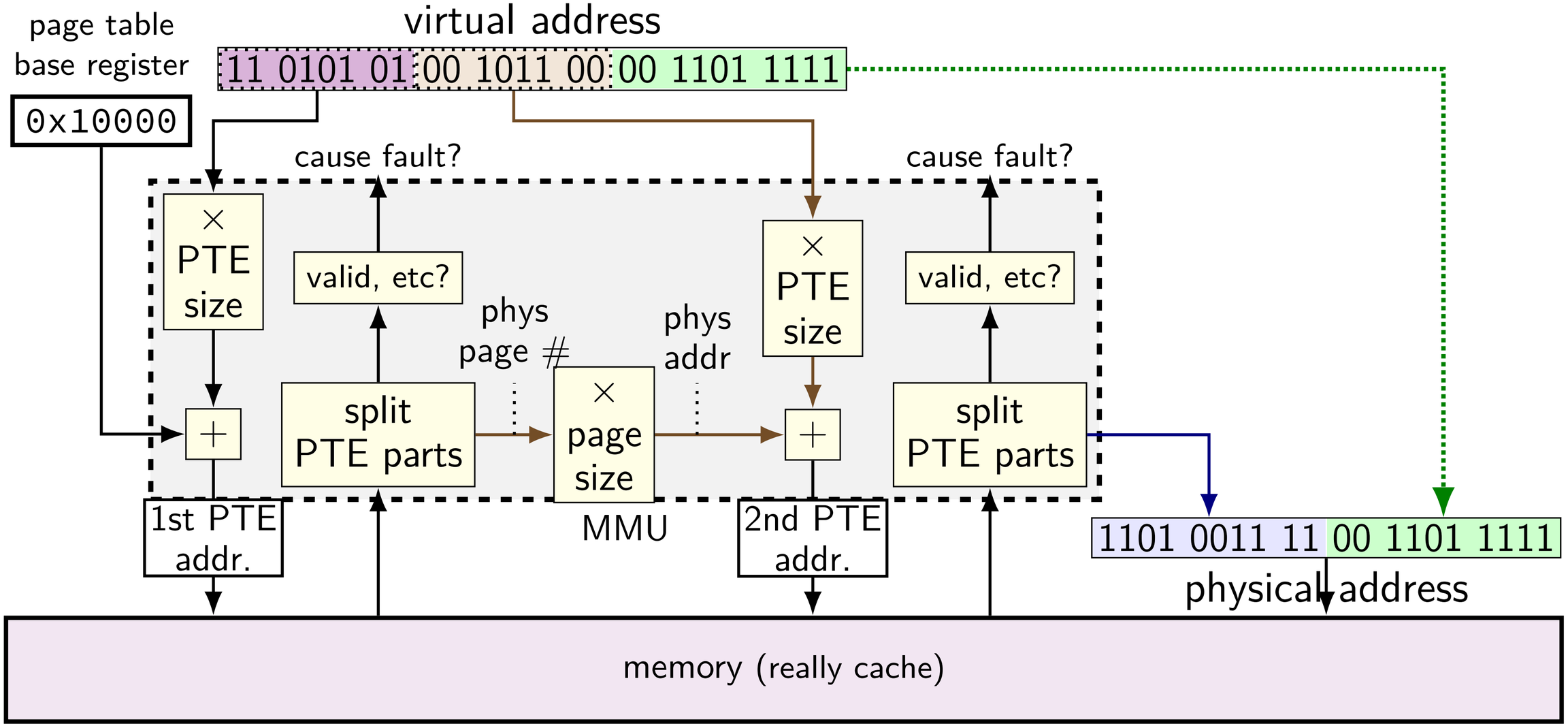
two-level page table lookup



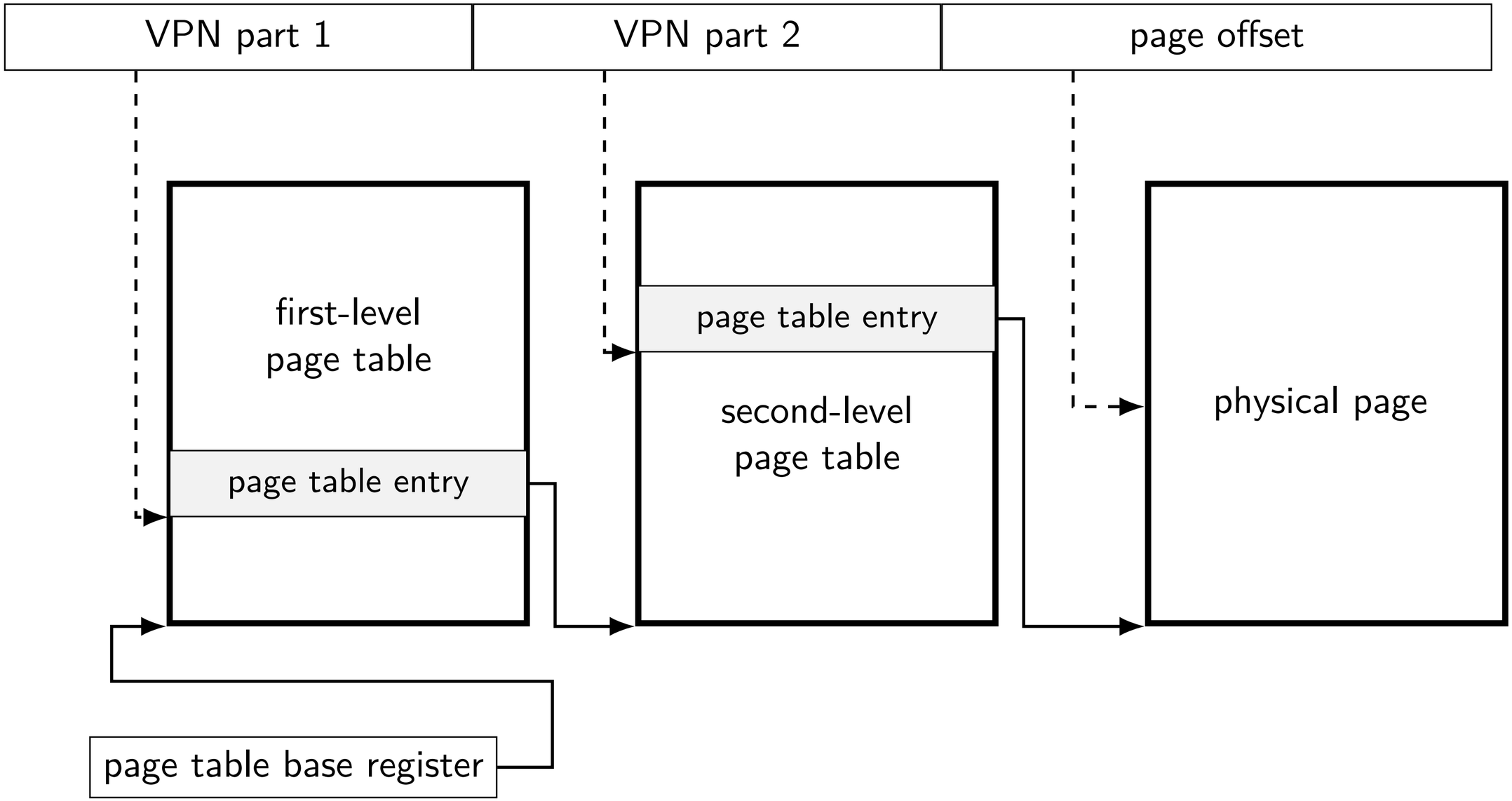
two-level page table lookup



two-level page table lookup



another view



cache accesses and multi-level PTs

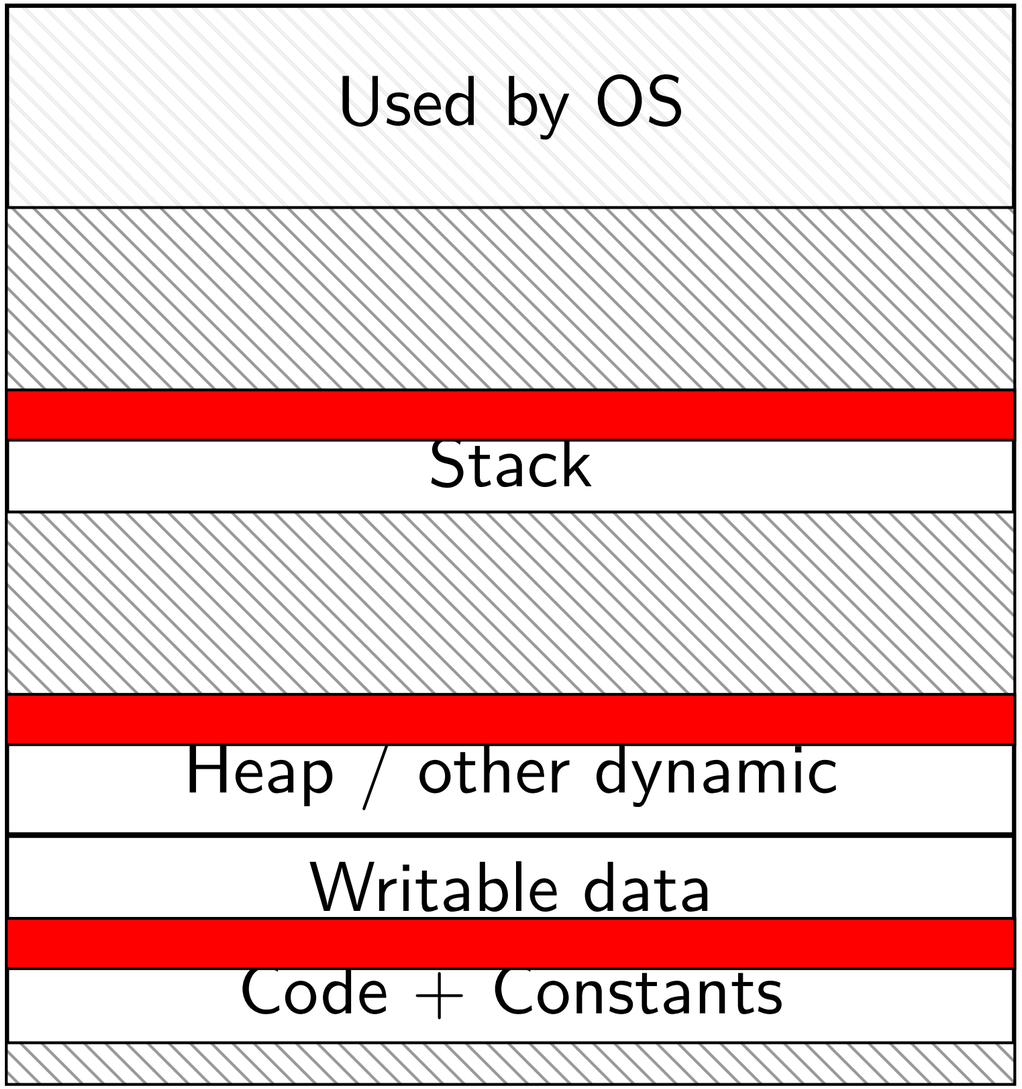
four-level page tables — five cache accesses per program memory access

L1 cache hits — typically a couple cycles each?

so add 8 cycles to each program memory access?

not acceptable

program memory active sets



`0xFFFF FFFF FFFF FFFF`

`0xFFFF 8000 0000 0000`

`0x7F...`

small areas of memory active at a time

one or two pages in each area?

`0x0000 0000 0040 0000`

page table entries and locality

page table entries have *excellent temporal locality*

typically one or two pages of the stack active

typically one or two pages of code active

typically one or two pages of heap/globals active

each page contains *whole functions*, arrays, stack frames, etc.

needed page table entries are *very small*

page table entry cache

caled a *TLB* (translation lookaside buffer)

(usually very small) cache of page table entries

L1 cache	**TLB**
physical addresses	virtual page numbers
bytes from memory	page table entries
tens of bytes per block	one page table entry per block
usually thousands of blocks	usually tens of entries

page table entry cache

caled a *TLB* (translation lookaside buffer)

(usually very small) cache of page table entries

L1 cache	**TLB**
physical addresses	*virtual page numbers*
bytes from memory	*page table entries*
tens of bytes per block	one page table entry per block
usually thousands of blocks	usually tens of entries

only caches the page table lookup itself
(generally) just entries from the last-level page tables

page table entry cache

caled a *TLB* (translation lookaside buffer)

(usually very small) cache of page table entries

L1 cache	**TLB**
physical addresses	*virtual page numbers*
bytes from memory	*page table entries*
tens of bytes per block	one page table entry per block
usually thousands of blocks	usually tens of entries

virtual page number divided into index + tag

page table entry cache

caled a *TLB* (translation lookaside buffer)

(usually very small) cache of page table entries

L1 cache	**TLB**
physical addresses	virtual page numbers
bytes from memory	page table entries
tens of bytes per block	*one page table entry per block*
usually thousands of blocks	usually tens of entries

not much spatial locality between page table entries
(they're used for kilobytes of data already)

page table entry cache

caled a *TLB* (translation lookaside buffer)

(usually very small) cache of page table entries

L1 cache	**TLB**
physical addresses	virtual page numbers
bytes from memory	page table entries
tens of bytes per block	*one page table entry per block*
usually thousands of blocks	usually tens of entries

`0` block offset bits

page table entry cache

caled a *TLB* (translation lookaside buffer)

(usually very small) cache of page table entries

L1 cache	**TLB**
physical addresses	virtual page numbers
bytes from memory	page table entries
tens of bytes per block	one page table entry per block
usually thousands of blocks	*usually tens of entries*

few active page table entries at a time
enables highly associative cache designs

TLB and multi-level page tables

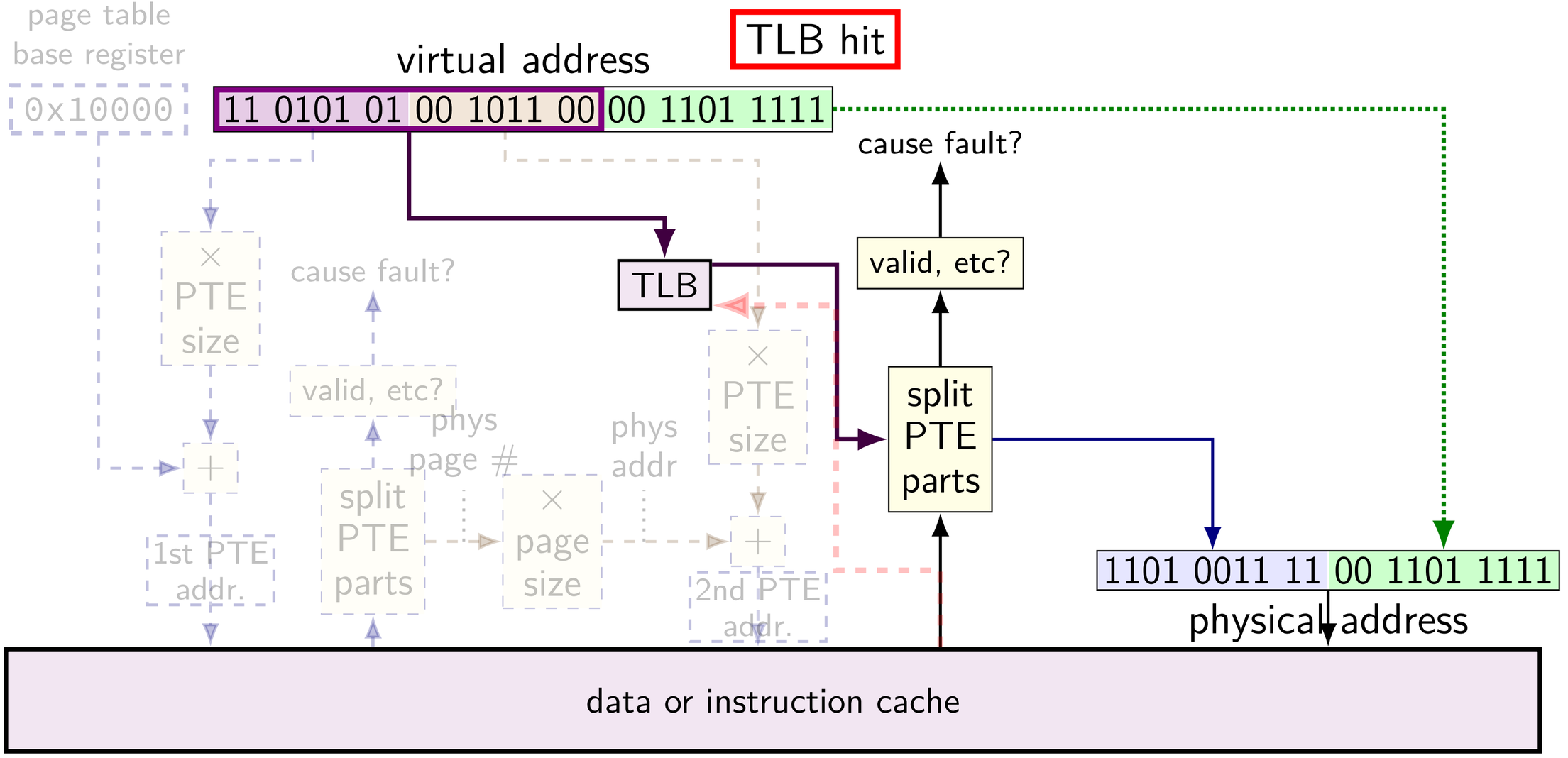
TLB caches *valid last-level page table entries*

doesn’t matter which last-level page table

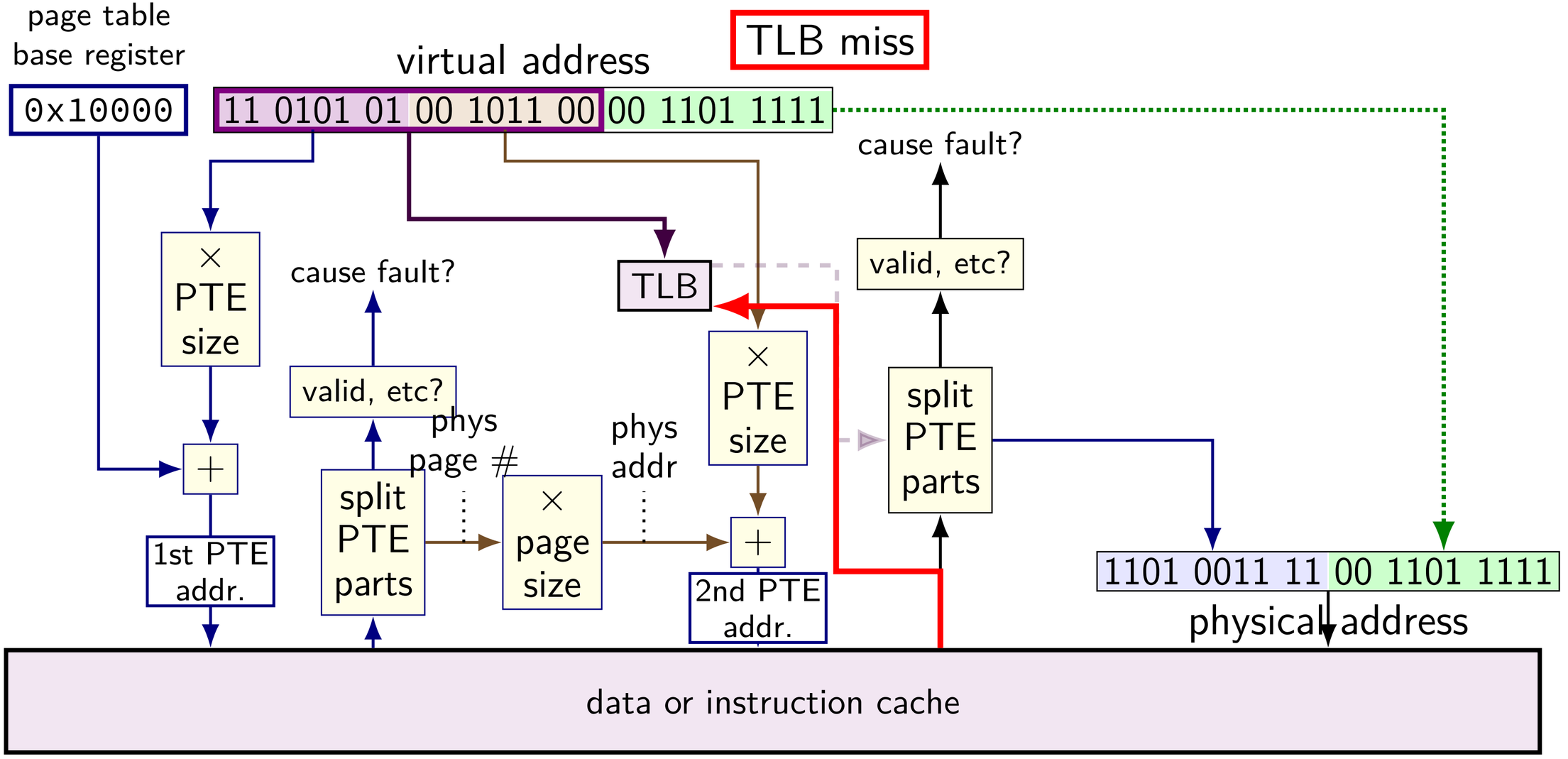
means TLB output can be used directly to form address

TLB and two-level lookup

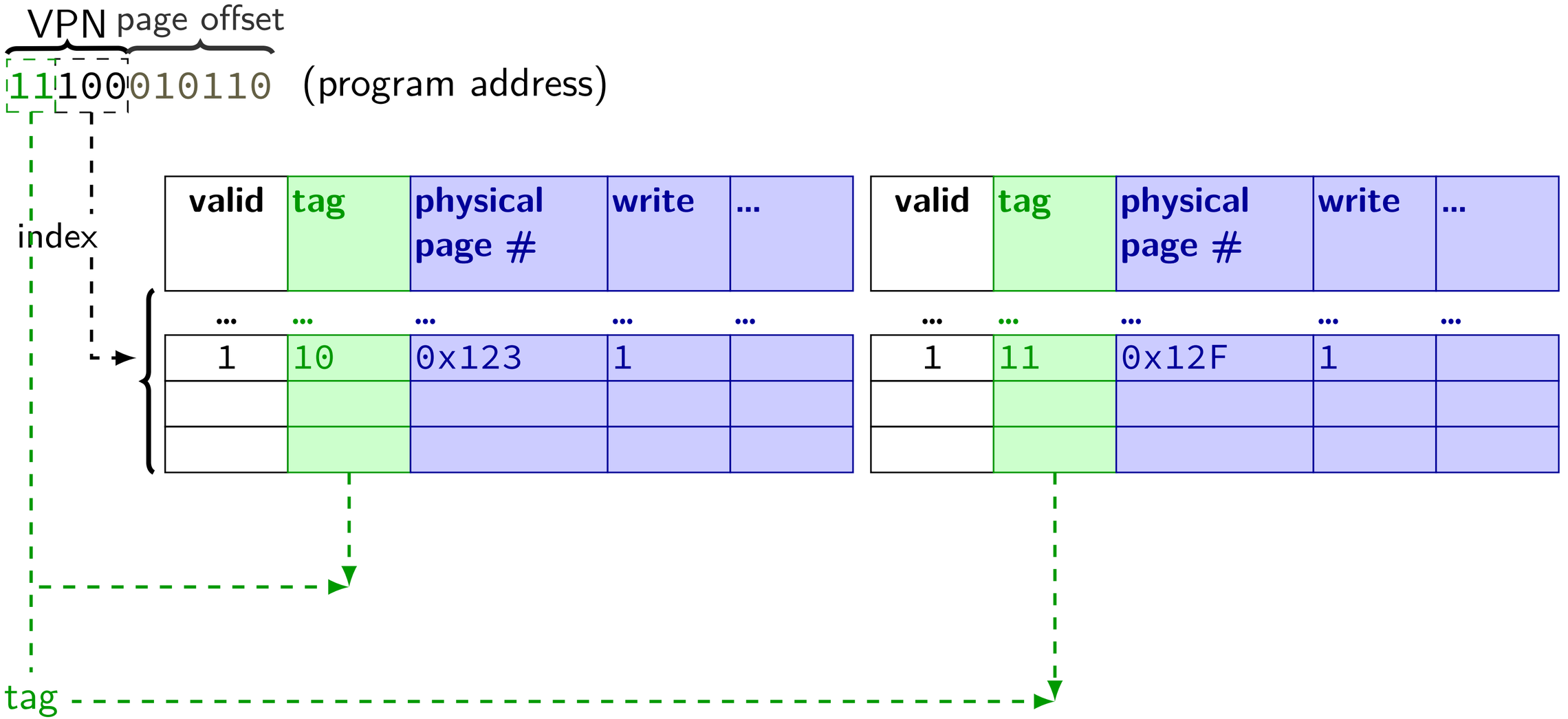
TLB and two-level lookup



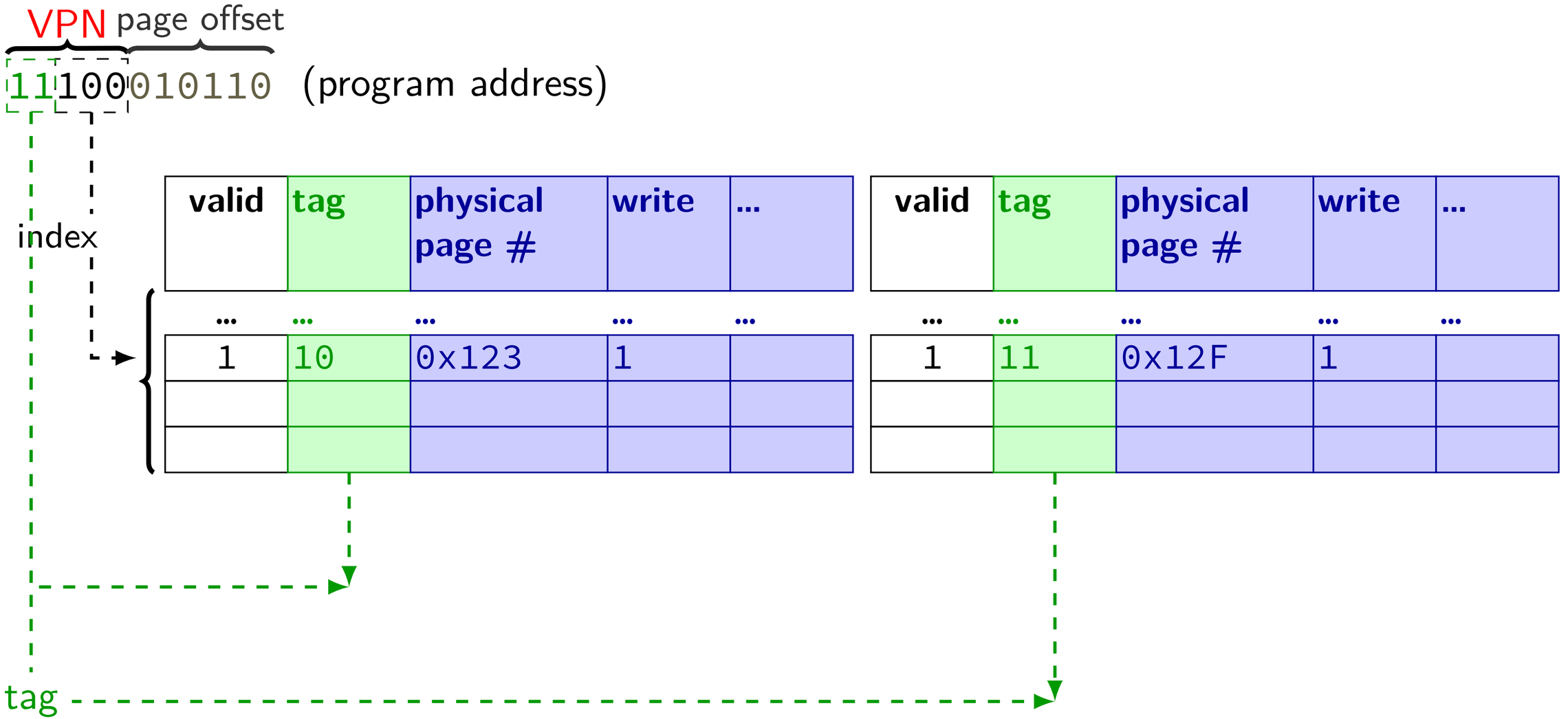
TLB and two-level lookup



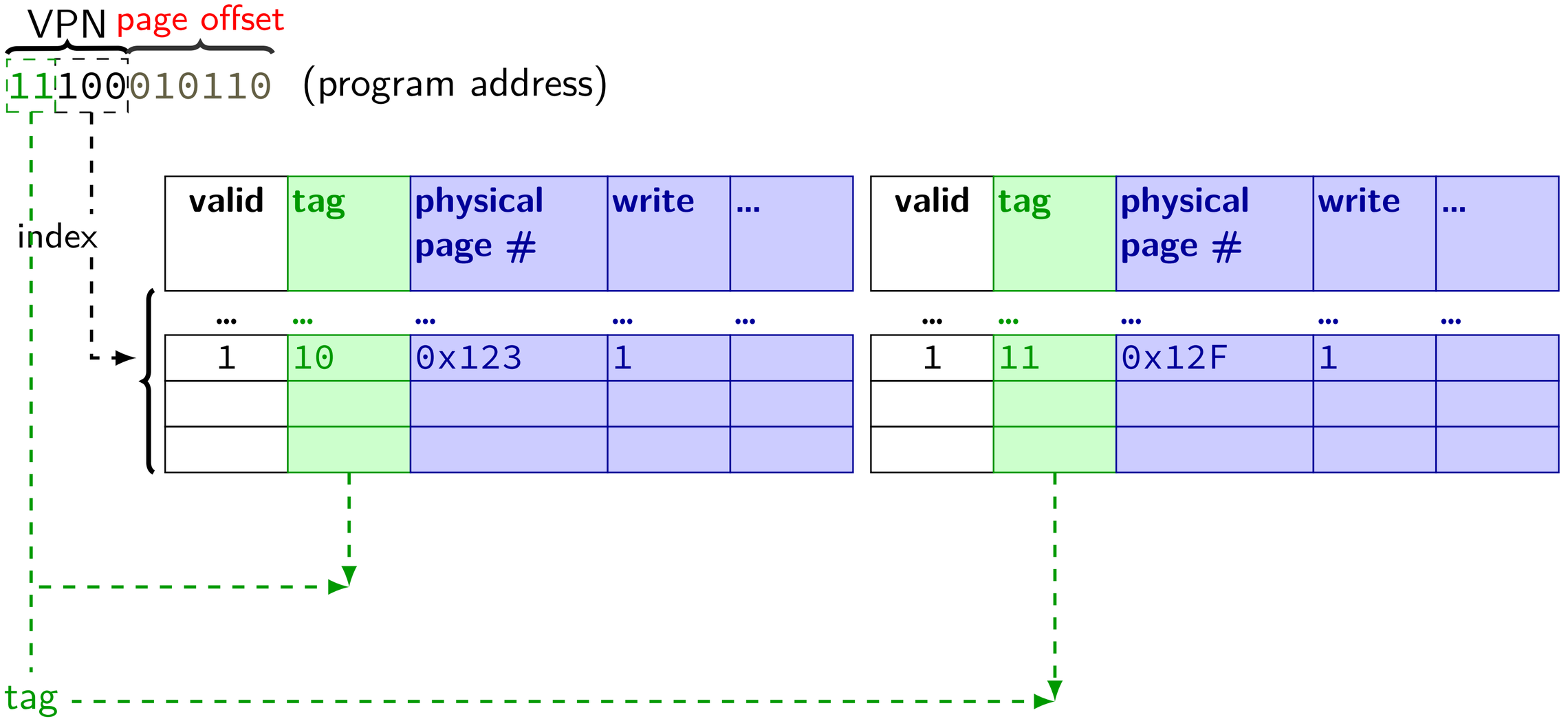
TLB organization (2-way set associative)



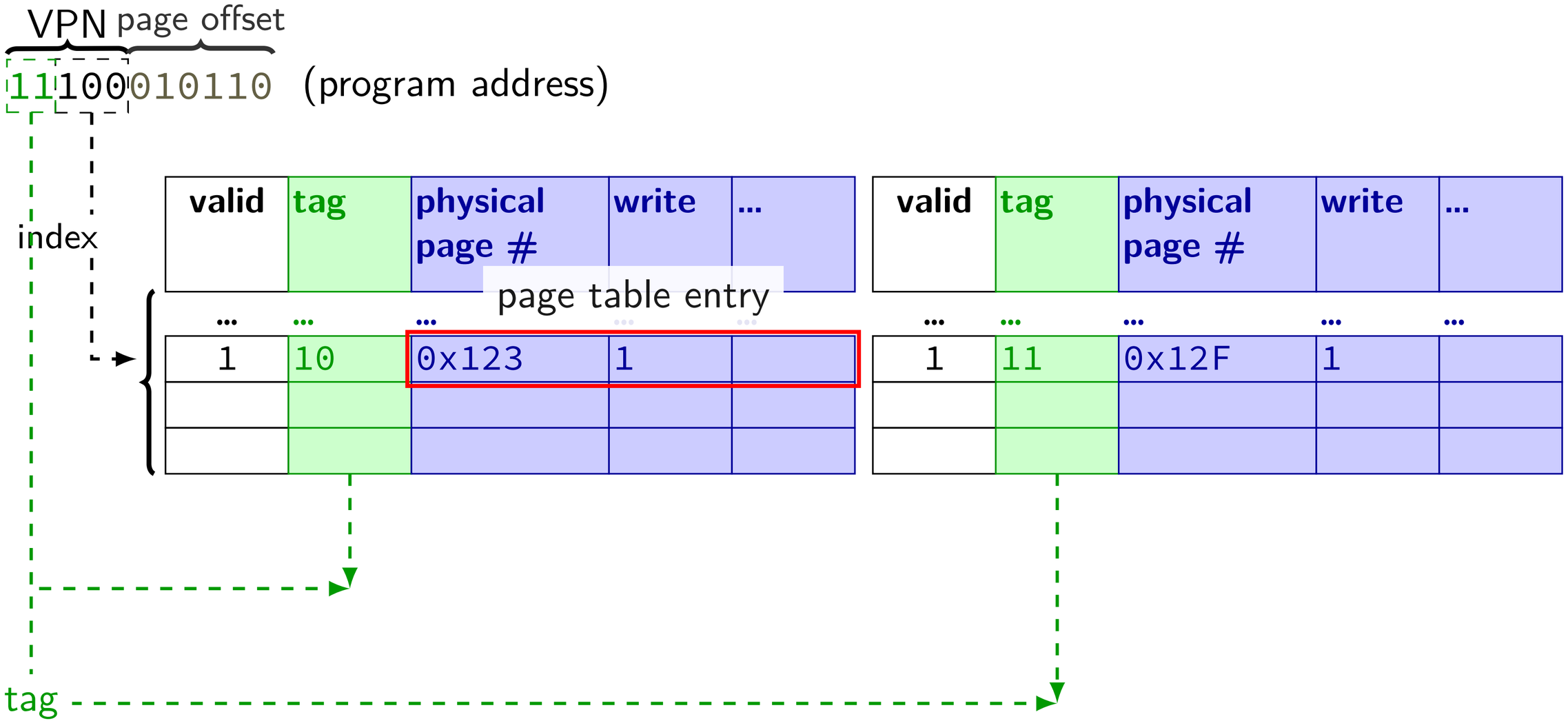
TLB organization (2-way set associative)



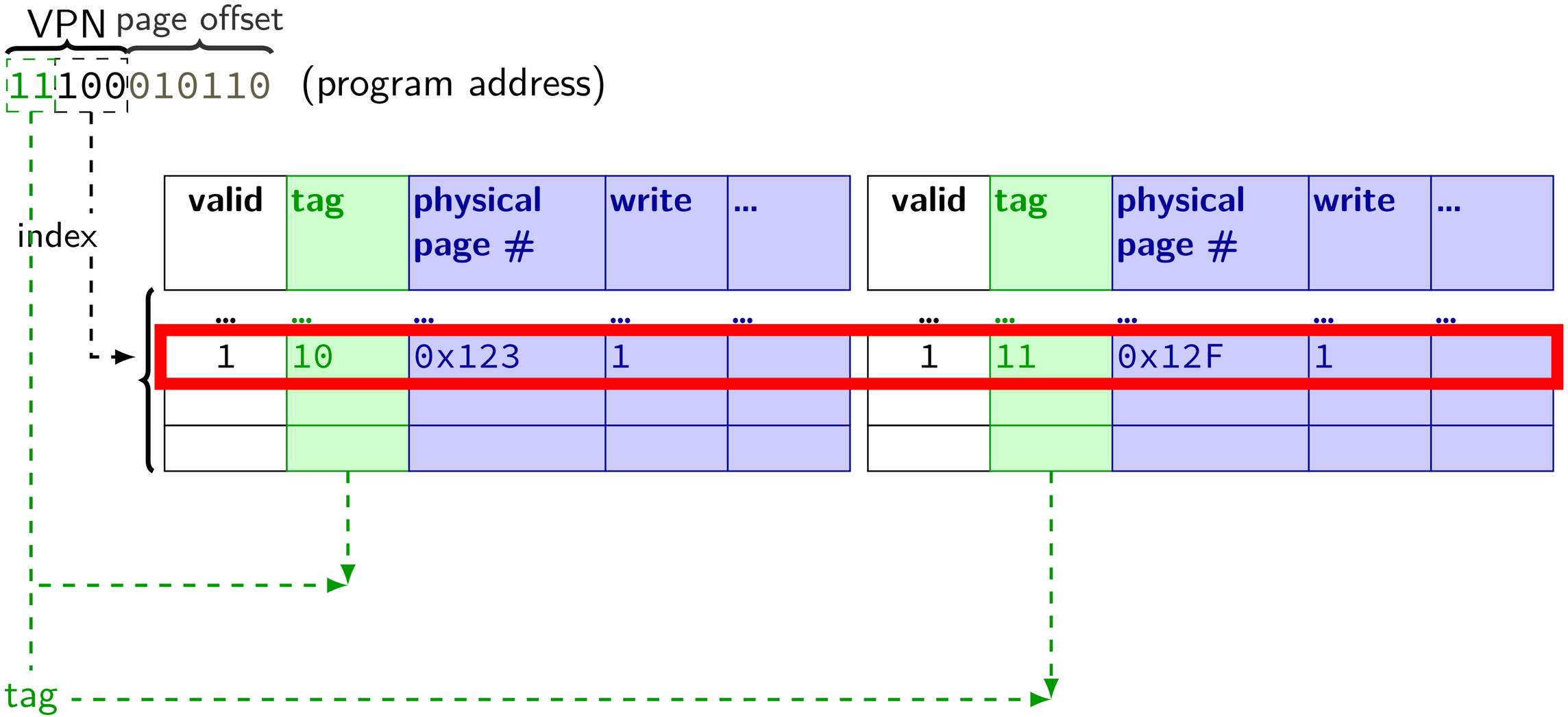
TLB organization (2-way set associative)



TLB organization (2-way set associative)



TLB organization (2-way set associative)



address splitting for TLBs (1)

my desktop:

4KB (2^{12} byte) pages; 48-bit virtual address

64-entry, 4-way L1 data TLB

TLB index bits?

TLB tag bits?

address splitting for TLBs (1)

my desktop:

4KB (2^{12} byte) pages; 48-bit virtual address

64-entry, 4-way L1 data TLB

TLB index bits? $64/4 = 16$ sets — 4 bits

TLB tag bits?

address splitting for TLBs (1)

my desktop:

4KB (2^{12} byte) pages; 48-bit virtual address

64-entry, 4-way L1 data TLB

TLB index bits? $64/4 = 16$ sets — 4 bits

TLB tag bits? $48 - 12 = 36$ bit virtual page number — $36 - 4 = 32$ bit TLB tag

address splitting for TLBs (2)

my desktop:

4KB (2^{12} byte) pages; 48-bit virtual address

1536-entry ($3 \cdot 2^9$), 12-way L2 TLB

TLB index bits?

TLB tag bits?

address splitting for TLBs (2)

my desktop:

4KB (2^{12} byte) pages; 48-bit virtual address

1536-entry ($3 \cdot 2^9$), 12-way L2 TLB

TLB index bits? $1536/12 = 128$ sets — 7 bits

TLB tag bits?

address splitting for TLBs (2)

my desktop:

4KB (2^{12} byte) pages; 48-bit virtual address

1536-entry ($3 \cdot 2^9$), 12-way L2 TLB

TLB index bits? $1536/12 = 128$ sets — 7 bits

TLB tag bits? $48 - 12 = 36$ bit virtual page number — $36 - 7 = 29$ bit TLB tag

exercise: TLB access pattern (setup)

4-entry, 2-way TLB, LRU replacement policy, initially empty

4096 byte pages

how many index bits?

TLB index of virtual address 0x12345?

exercise: TLB access pattern

4-entry, 2-way TLB, LRU replacement policy, initially empty

4096 byte pages

type	virtual	physical	result	set 0	set 1
read	0x440030	0x554030			
write	0x440034	0x554034			
read	0x7FFFE008	0x556008			
read	0x7FFFE000	0x556000			
read	0x7FFFDFF8	0x5F8FF8			
read	0x664080	0x5F9080			
read	0x440038	0x554038			
write	0x7FFFDFF0	0x5F8FF0			

which are TLB hits? which are TLB misses? final contents of TLB?

solution: TLB access pattern

type	virtual	physical	result	set 0	set 1
read	0x440030	0x554030	miss	0x440	
write	0x440034	0x554034	hit	0x440	
read	0x7FFFE008	0x556008	miss	0x440, 0x7FFFE	
read	0x7FFFE000	0x556000	hit	0x440, 0x7FFFE	
read	0x7FFFDFF8	0x5F8FF8	miss	0x440, 0x7FFFE	0x7FFFD
read	0x664080	0x5F9080	miss	0x664, 0x7FFFE	0x7FFFD
read	0x440038	0x554038	miss	0x664, 0x440	0x7FFFD
write	0x7FFFDFF0	0x5F8FF0	hit	0x664, 0x440	0x7FFFD

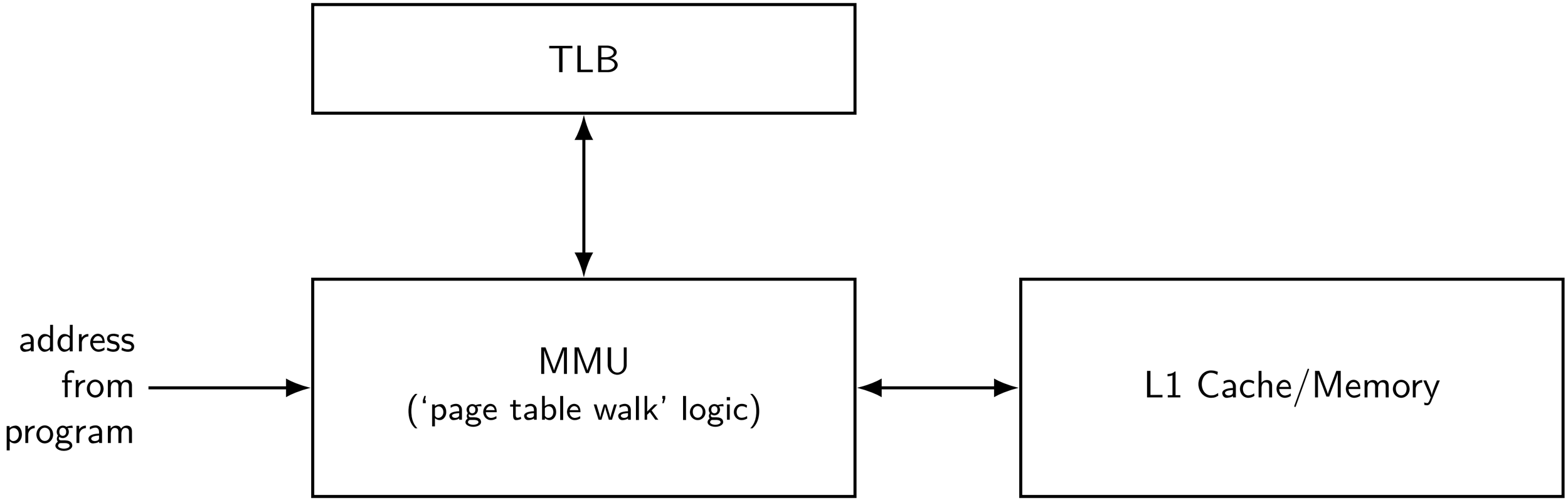
solution: TLB access pattern

type	virtual	physical	result	set 0	set 1
read	0x440030	0x554030	miss	0x440	
write	0x440034	0x554034	hit	0x440	
read	0x7FFFE008	0x556008	miss	0x440, 0x7FFFE	
read	0x7FFFE000	0x556000	hit	0x440, 0x7FFFE	
read	0x7FFFDFF8	0x5F8FF8	miss	0x440, 0x7FFFE	0x7FFFD
read	0x664080	0x5F9080	miss	0x664, 0x7FFFE	0x7FFFD
read	0x440038	0x554038	miss	0x664, 0x440	0x7FFFD
write	0x7FFFDFF0	0x5F8FF0	hit	0x664, 0x440	0x7FFFD

set idx	V	tag	physical page	write?	user?	…	LRU?
0	1	0x00220 (0x00440 >> 1)	0x554	1	1	…	no
	1	0x00322 (0x00664 >> 1)	0x5F9	1	1	…	yes
1	1	0x3FFFF (0x7FFFD >> 1)	0x554	1	1	…	no
	0					…	yes

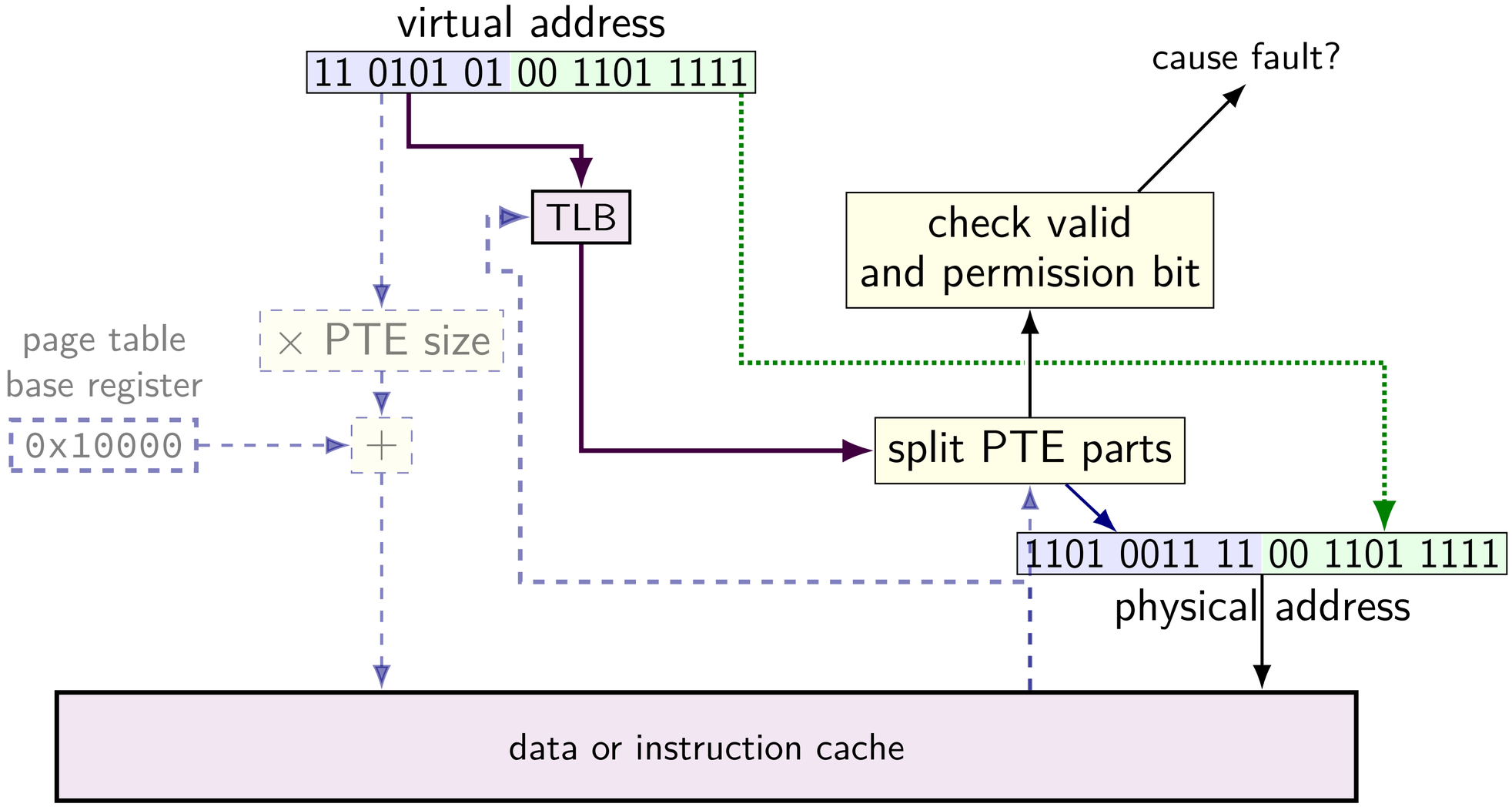
backup slides

TLB and the MMU (1)

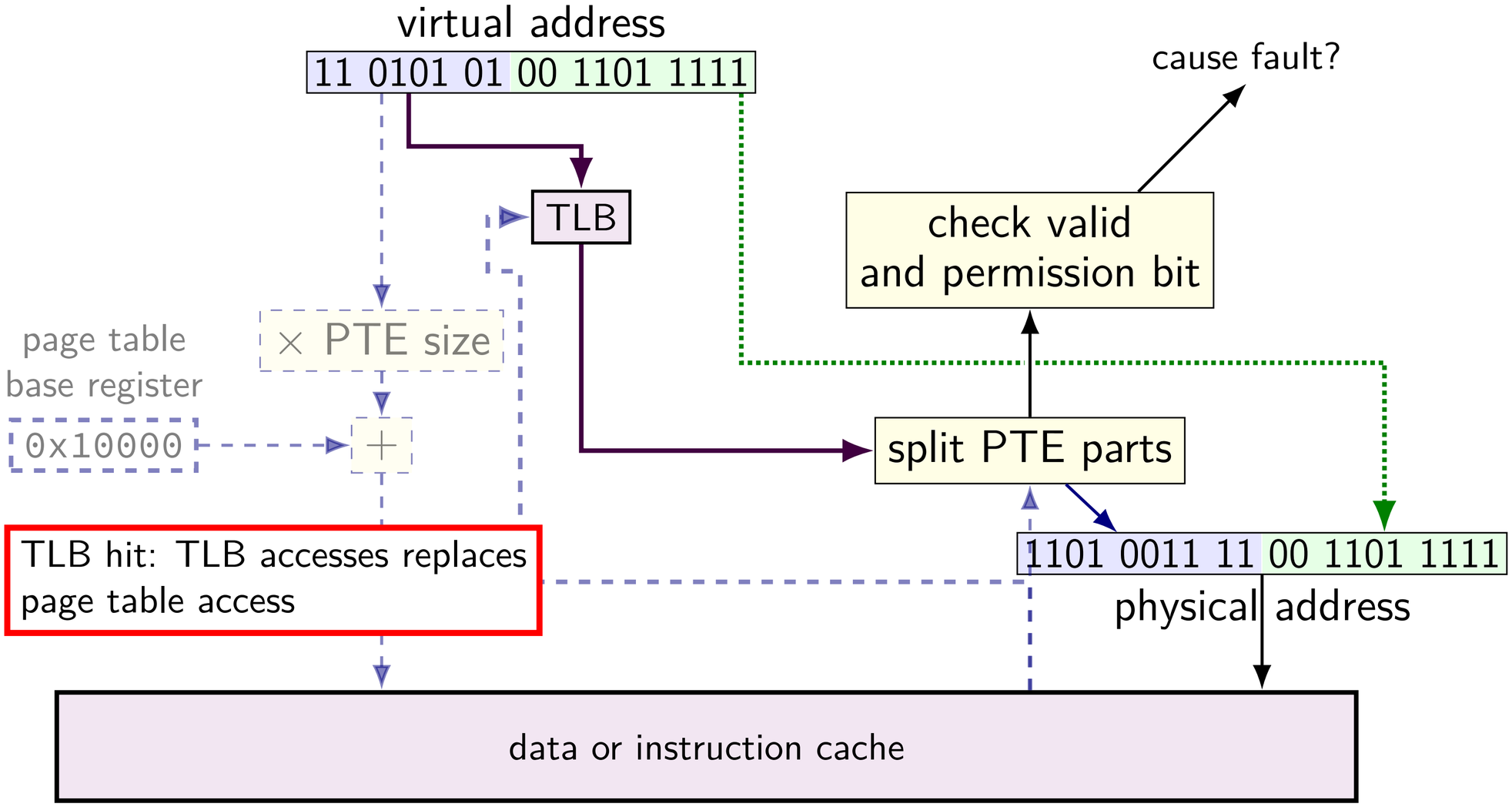


TLB and the MMU (2)

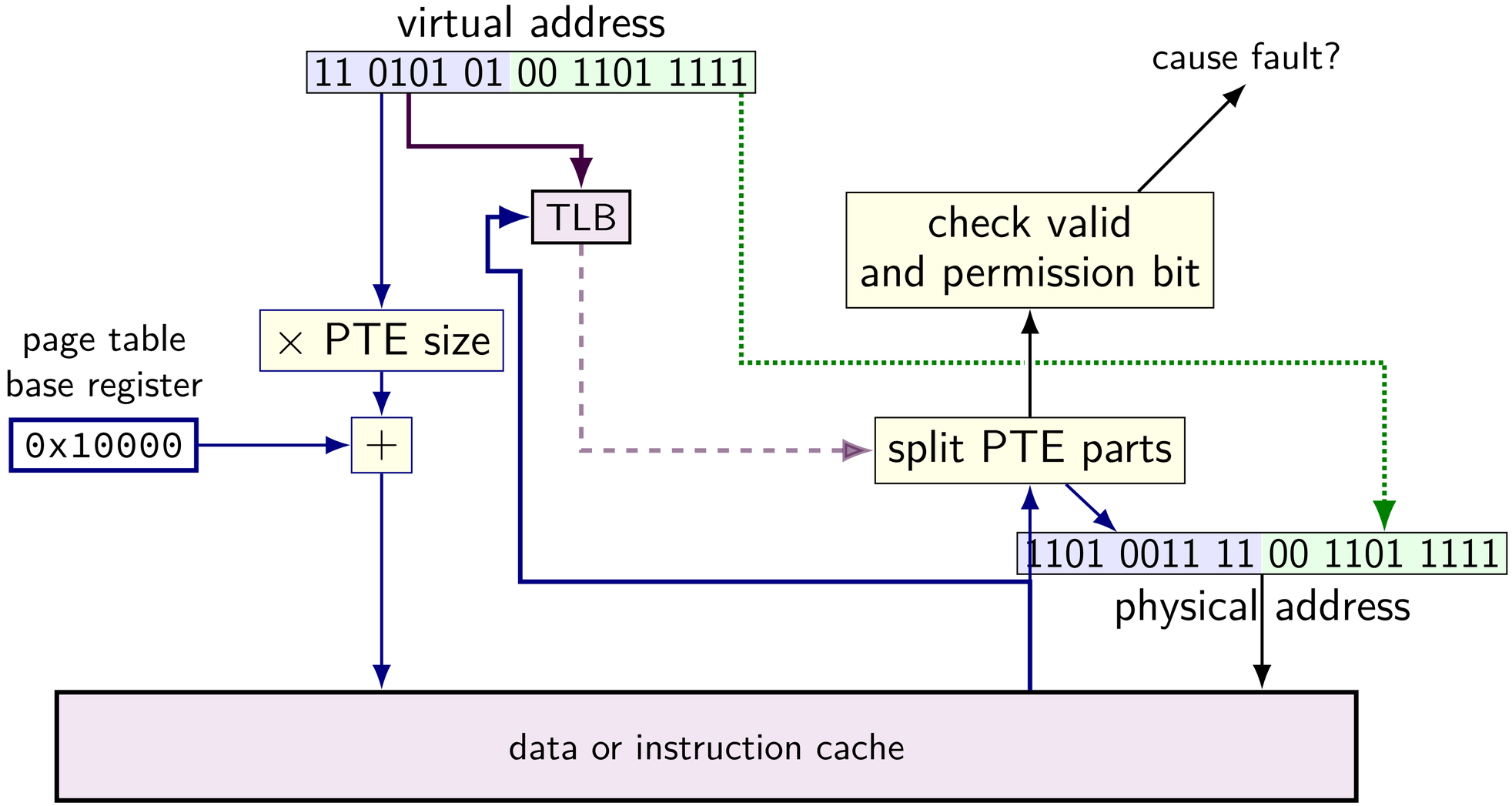
TLB and the MMU (2)



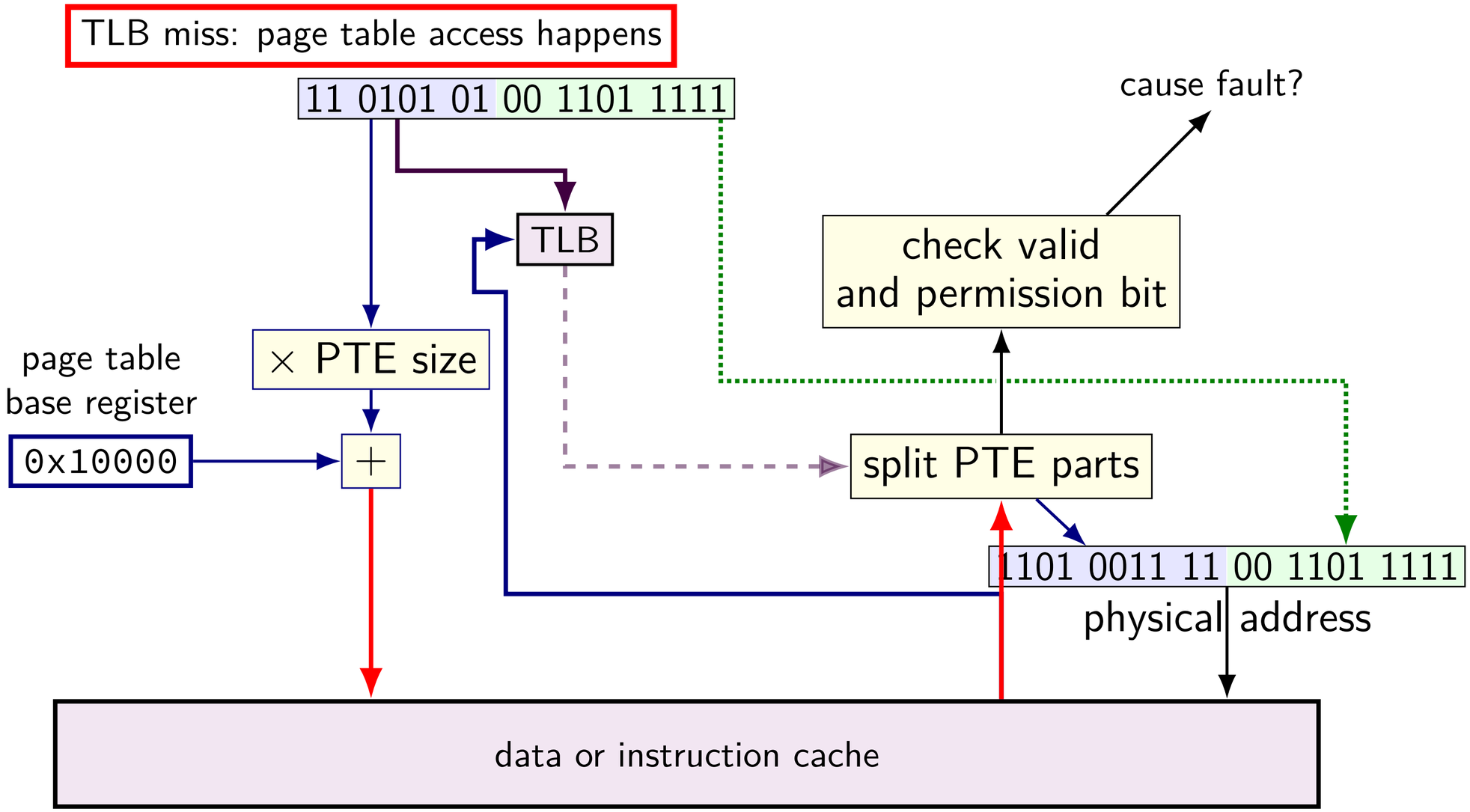
TLB and the MMU (2)



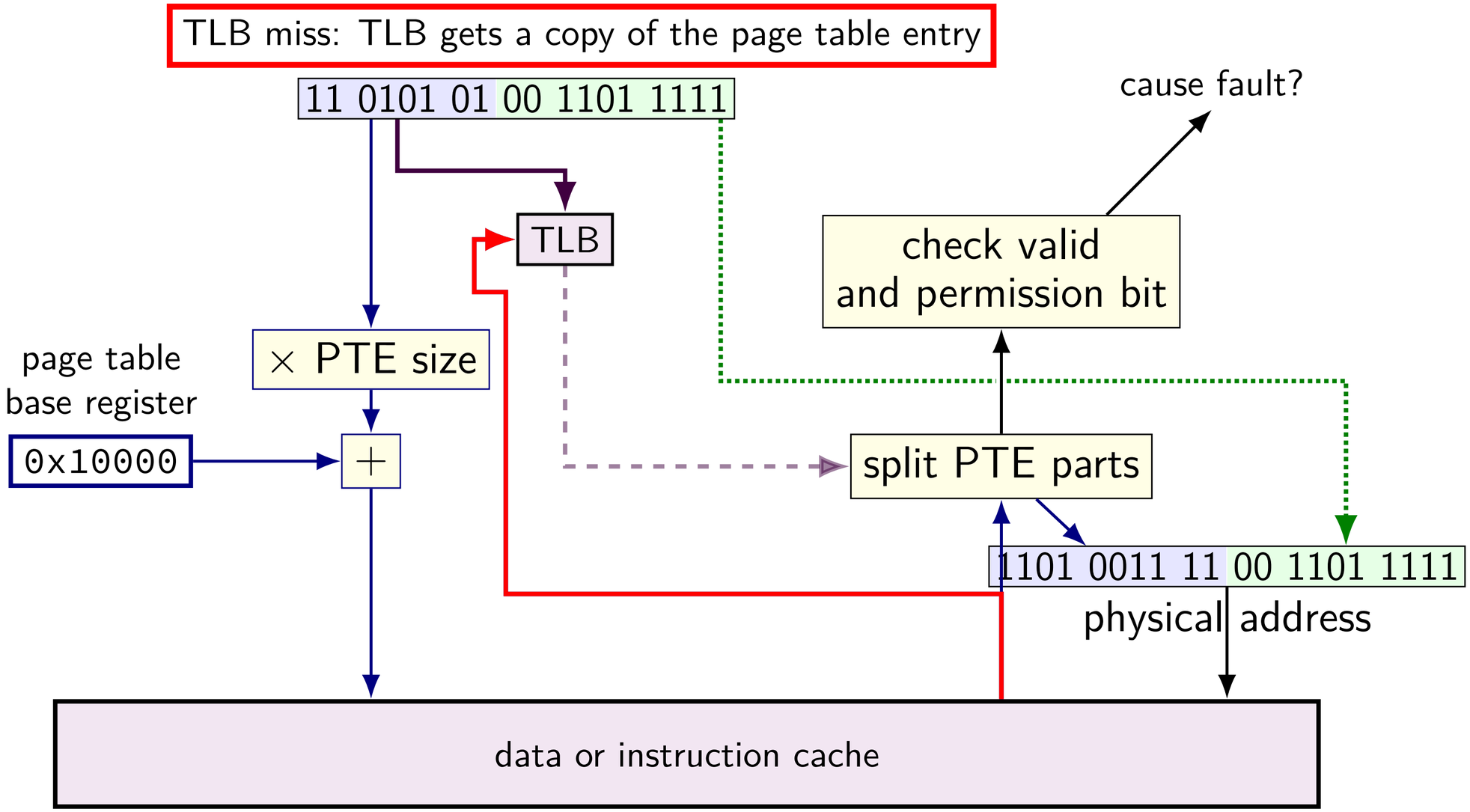
TLB and the MMU (2)



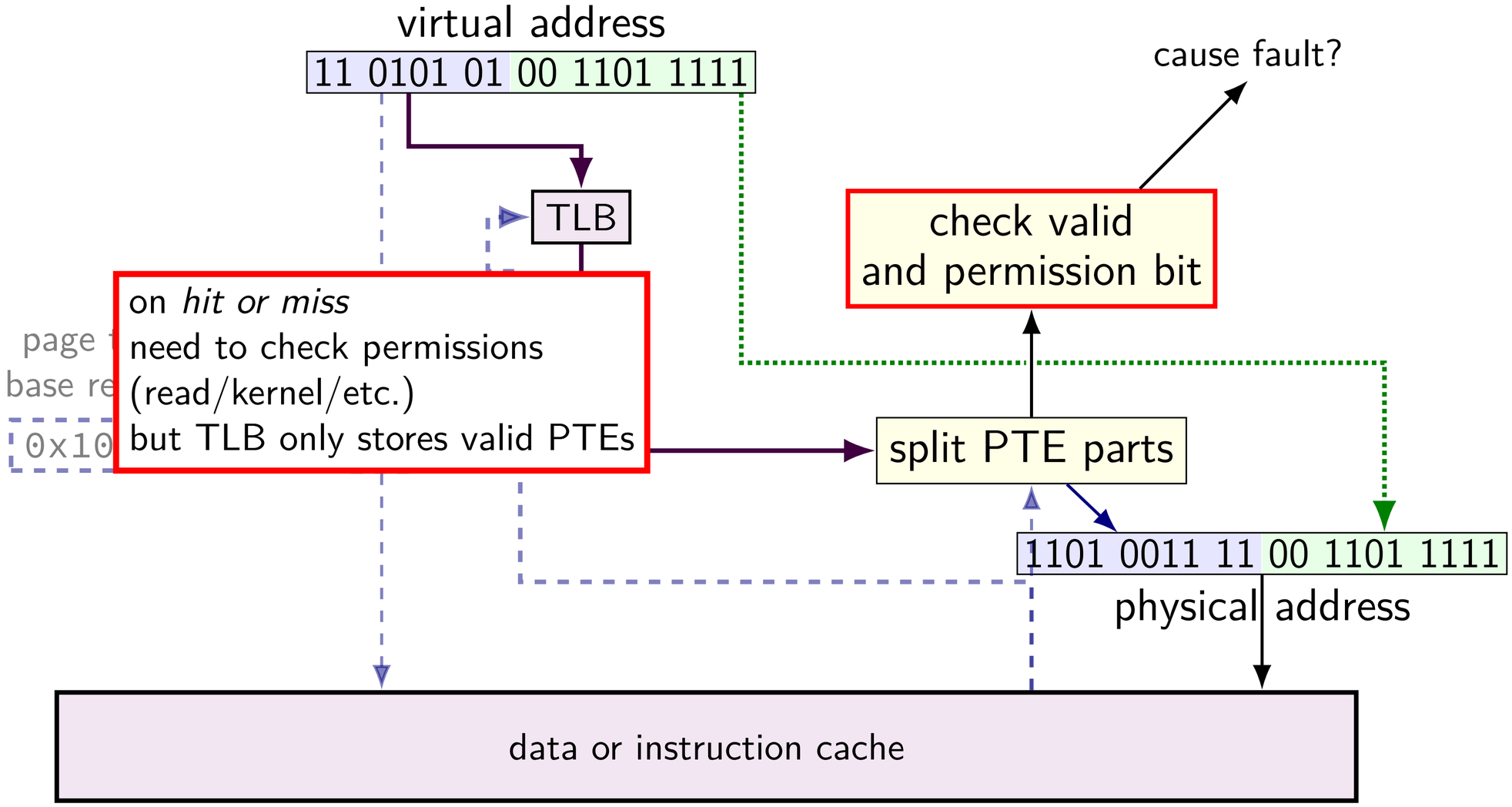
TLB and the MMU (2)



TLB and the MMU (2)



TLB and the MMU (2)



changing page tables

what happens to TLB when page table base pointer is changed?

- e.g. context switch

most entries in TLB refer to things from *wrong process*

- oops — read from the wrong process's stack?

option 1: *invalidate* all TLB entries

- side effect on "change page table base register" instruction

option 2: TLB entries contain process ID

- set by OS (special register)
- checked by TLB in addition to TLB tag, valid bit

editing page tables

what happens to TLB when OS changes a page table entry?

most common choice: has to be handled *in software*

invalid to valid — nothing needed

- TLB doesn't contain invalid entries
- MMU will check memory again

valid to invalid — *OS needs to tell processor* to invalidate it

- special instruction (x86: `invlpg`)

valid to other valid — *OS needs to tell processor* to invalidate it

changing page tables

what happens to TLB when page table base pointer is changed?

- e.g. context switch

most entries in TLB refer to things from *wrong process*

- oops — read from the wrong process's stack?

option 1: *invalidate* all TLB entries

- side effect on "change page table base register" instruction

option 2: TLB entries contain process ID

- set by OS (special register)
- checked by TLB in addition to TLB tag, valid bit

editing page tables

what happens to TLB when OS changes a page table entry?

most common choice: has to be handled *in software*

invalid to valid — nothing needed

- TLB doesn't contain invalid entries
- MMU will check memory again

valid to invalid — *OS needs to tell processor* to invalidate it

- special instruction (x86: `invlpg`)

valid to other valid — *OS needs to tell processor* to invalidate it

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

page table entry

set idx	V	tag	physical page	write?	user?	···
0	0					•••
1	0					•••

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	
0x11038	0001 000 1	0xFFF038	
0x11040	0001 000 1	0xFFF040	
0x7CFF0	0111 110 0	0x3100F0	
0x11048	0001 000 1	0xFFF048	
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

page table entry

set idx	V	tag	physical page	write?	user?	···
0	0					•••
1	1					•••

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	
0x11040	0001 000 1	0xFFF040	
0x7CFF0	0111 110 0	0x3100F0	
0x11048	0001 000 1	0xFFF048	
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

			page table entry			
set idx	V	tag	physical page	write?	user?	···
0	0					···
1	1	0001000	0xFFF	1	1	···

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	
0x11040	0001 000 1	0xFFF040	
0x7CFF0	0111 110 0	0x3100F0	
0x11048	0001 000 1	0xFFF048	
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

			page table entry			
set idx	V	tag	physical page	write?	user?	···
0	0					···
1	1	0001000	0xFFF	1	1	···

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	
0x7CFF0	0111 110 0	0x3100F0	
0x11048	0001 000 1	0xFFF048	
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

	page table entry					
set idx	V	tag	physical page	write?	user?	···
0	0					···
1	1	0001000	0xFFF	1	1	···

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	
0x11048	0001 000 1	0xFFF048	
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

			page table entry			
set idx	V	tag	physical page	write?	user?	···
0	0					•••
1	1	0001000	0xFFF	1	1	•••

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	
0x11048	0001 000 1	0xFFF048	
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

page table entry

set idx	V	tag	physical page	write?	user?	···
0	1	0111110	0x310	1	1	•••
1	1	0001000	0xFFF	1	1	•••

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	miss
0x11048	0001 000 1	0xFFF048	
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

	page table entry				
set idx	V	tag	physical page	write?	user?	...
0	1	0111110	0x310	1	1	...
1	1	0001000	0xFFF	1	1	...

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	miss
0x11048	0001 000 1	0xFFF048	hit
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

	page table entry				
set idx	V	tag	physical page	write?	user?	…
0	1	0111110	0x310	1	1	…
1	1	0001000	0xFFF	1	1	…

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	miss
0x11048	0001 000 1	0xFFF048	hit
0x7CFE8	0111 110 0	0x310FE8	
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

	page table entry				
set idx	V	tag	physical page	write?	user?	···
0	1	0111110	0x310	1	1	···
1	1	0001000	0xFFF	1	1	···

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	miss
0x11048	0001 000 1	0xFFF048	hit
0x7CFE8	0111 110 0	0x310FE8	hit
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

	page table entry				
set idx	V	tag	physical page	write?	user?	...
0	1	0111110	0x310	1	1	...
1	1	0001000	0xFFF	1	1	...

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	miss
0x11048	0001 000 1	0xFFF048	hit
0x7CFE8	0111 110 0	0x310FE8	hit
0x30000	0011 000 0	0x8FF000	
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

page table entry

set idx	V	tag	physical page	write?	user?	···
0	1	*0011000*	*0x8FF*	1	1	···
1	1	0001000	0xFFF	1	1	···

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	miss
0x11048	0001 000 1	0xFFF048	hit
0x7CFE8	0111 110 0	0x310FE8	hit
0x30000	0011 000 0	0x8FF000	miss
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

page table entry

set idx	V	tag	physical page	write?	user?	···
0	1	0011000	0x8FF	1	1	···
1	1	0001000	0xFFF	1	1	···

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	miss
0x11048	0001 000 1	0xFFF048	hit
0x7CFE8	0111 110 0	0x310FE8	hit
0x30000	0011 000 0	0x8FF000	miss
0x7CFE0	0111 110 0	0x310FE0	

TLB access pattern example

2-entry, direct-mapped TLB, 4096 byte pages
\setlength{}\fboxsep{0.25mm}

	page table entry				
set idx	V	tag	physical page	write?	user?	…
0	1	*0111110*	*0x310*	1	1	…
1	1	0001000	0xFFF	1	1	…

virtual	VPN (binary)	physical	hit/miss?
0x11030	0001 000 1	0xFFF030	miss
0x11038	0001 000 1	0xFFF038	hit
0x11040	0001 000 1	0xFFF040	hit
0x7CFF0	0111 110 0	0x3100F0	miss
0x11048	0001 000 1	0xFFF048	hit
0x7CFE8	0111 110 0	0x310FE8	hit
0x30000	0011 000 0	0x8FF000	miss
0x7CFE0	0111 110 0	0x310FE0	miss

TLB example: address splitting

16-bit virtual addresses

64-byte pages

8-entry, 2-way TLB

TLB access pattern

64-byte pages

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	0	????	—	0	????	—	?
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	0	????	—	0	????	—	?

address (hex)	hit?
000100000000 (100)	
110100000001 (D01)	
000100001010 (10A)	
110100100001 (D21)	
000011111100 (0FC)	
110011111000 (CF8)	
111100101000 (F23)	

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	0	????	—	0	????	—	?
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	0	????	—	0	????	—	?

address (hex)	hit?
000100000000 (100)	
110100000001 (D01)	
000100001010 (10A)	
110100100001 (D21)	
000011111100 (0FC)	
110011111000 (CF8)	
111100101000 (F23)	

VPN

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	0	????	—	0	????	—	?
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	0	????	—	0	????	—	?

address (hex)	hit?
000100000000 (100)	
110100000001 (D01)	
000100001010 (10A)	
110100100001 (D21)	
000011111100 (0FC)	
110011111000 (CF8)	
111100101000 (F23)	

VPN

tag index

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	1	0001	for VPN 000100	0	????		way 1
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	0	????	—	0	????	—	?

address (hex)	hit?
000100000000 (100)	miss
110100000001 (D01)	
000100001010 (10A)	
110100100001 (D21)	
000011111100 (0FC)	
110011111000 (CF8)	
111100101000 (F23)	

VPN

tag index

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	1	0001	for VPN 000100	1	1101	for VPN 110100	way 0
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	0	????	—	0	????	—	?

address (hex)	hit?
000100000000 (100)	miss
110100000001 (D01)	miss
000100001010 (10A)	
110100100001 (D21)	
000011111100 (0FC)	
110011111000 (CF8)	
111100101000 (F23)	

VPN

tag index

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	1	0001	for VPN 000100	1	1101	for VPN 110100	way 1
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	0	????	—	0	????	—	?

address (hex)	hit?
000100000000 (100)	miss
110100000001 (D01)	miss
000100001010 (10A)	hit
110100100001 (D21)	
000011111100 (0FC)	
110011111000 (CF8)	
111100101000 (F23)	

VPN

tag index

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	1	0001	for VPN 000100	1	1101	for VPN 110100	way 0
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	0	????	—	0	????	—	?

address (hex)	hit?
000100000000 (100)	miss
110100000001 (D01)	miss
000100001010 (10A)	hit
110100100001 (D21)	hit
000011111100 (0FC)	
110011111000 (CF8)	
111100101000 (F23)	

VPN

tag index

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	1	0001	for VPN 000100	1	1101	for VPN 110100	way 0
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	1	0000	for VPN 000011		????	—	way 1

address (hex)	hit?
000100000000 (100)	miss
110100000001 (D01)	miss
000100001010 (10A)	hit
110100100001 (D21)	hit
000011111100 (0FC)	miss
110011111000 (CF8)	
111100101000 (F23)	

VPN

tag index

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	1	0001	for VPN 000100	1	1101	for VPN 110100	way 0
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	1	0000	for VPN 000011	1	1100	for VPN 110011	way 0

address (hex)	hit?
000100000000 (100)	miss
110100000001 (D01)	miss
000100001010 (10A)	hit
110100100001 (D21)	hit
000011111100 (0FC)	miss
110011111000 (CF8)	miss
111100101000 (F23)	

VPN

tag index

TLB access pattern

64-byte pages

index	V	tag	PTE	V	tag	PTE	LRU
00	1	1111	for VPN 111100	1	1101	for VPN 110100	way 1
01	0	????	—	0	????	—	?
10	0	????	—	0	????	—	?
11	1	0000	for VPN 000011	1	1100	for VPN 110011	way 0

address (hex)	hit?
000100000000 (100)	miss
110100000001 (D01)	miss
000100001010 (10A)	hit
110100100001 (D21)	hit
000011111100 (0FC)	miss
110011111000 (CF8)	miss
111100101000 (F23)	miss

VPN

tag index

counting TLB misses

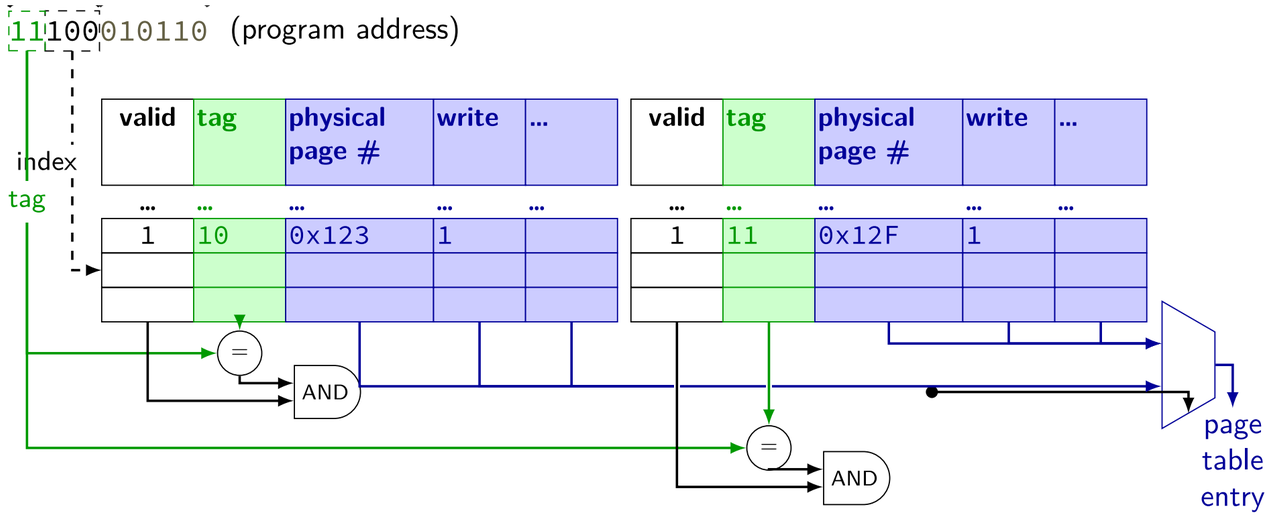
4-entry 2-way set associative TLB

LRU replacement policy

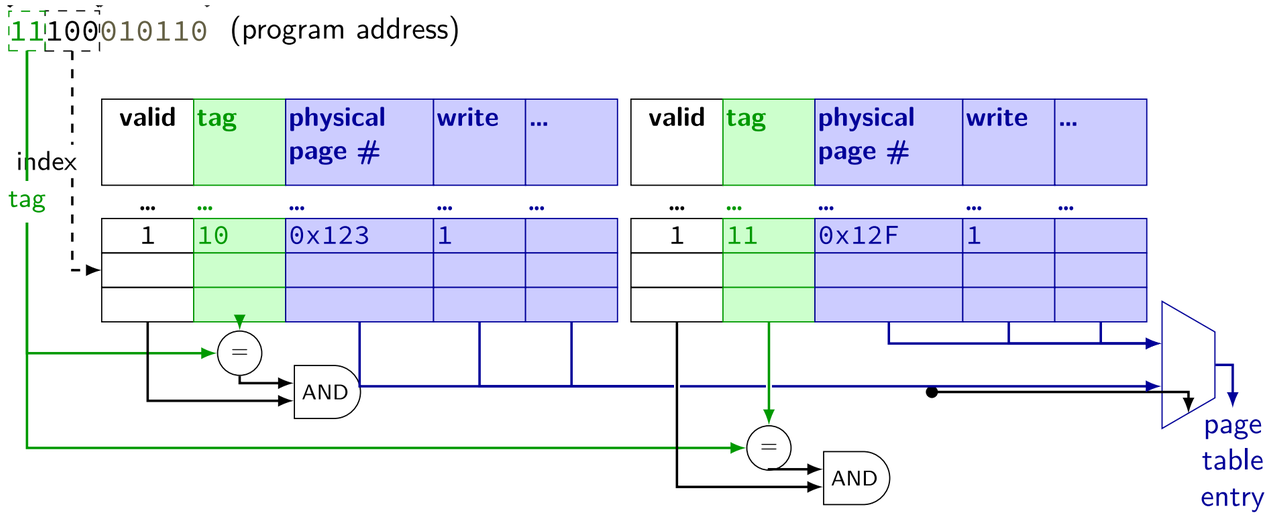
8-bit virtual address, 4-bit VPN

TLB organization (2-way set associative)

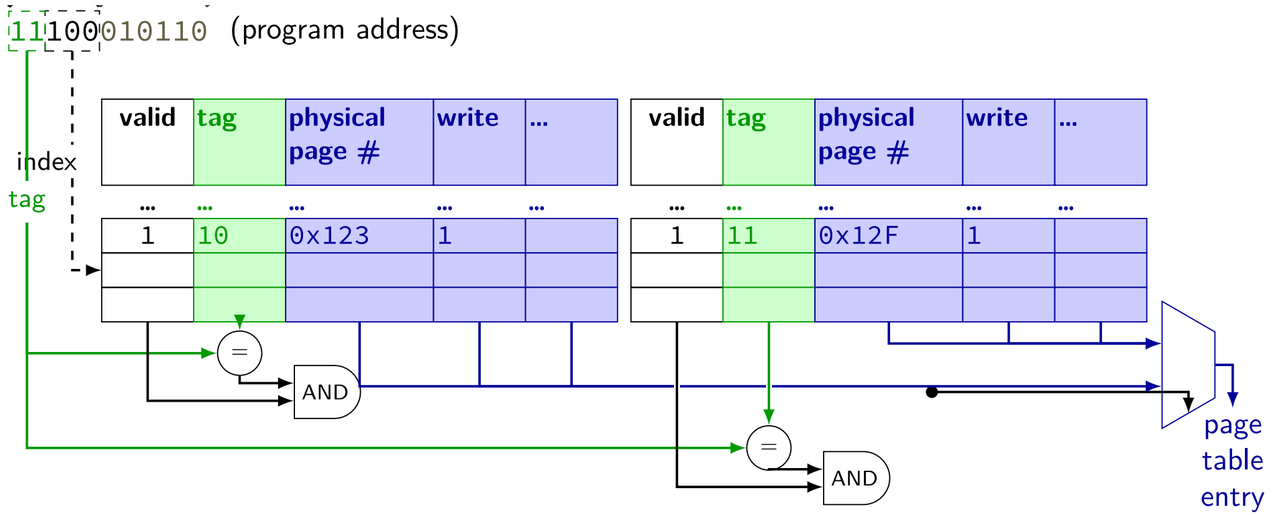
TLB organization (2-way set associative)



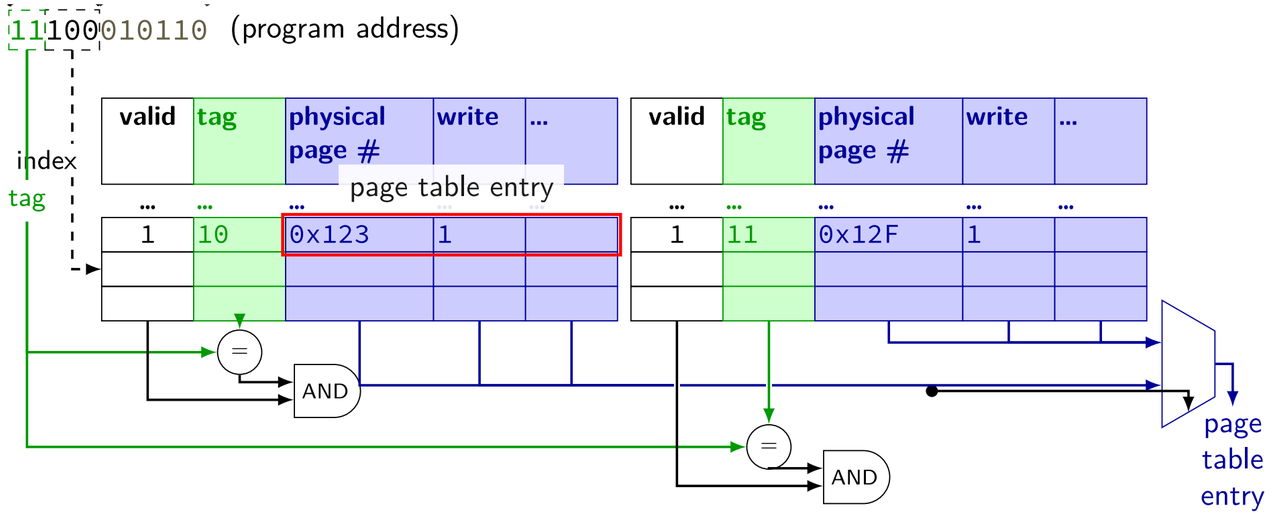
TLB organization (2-way set associative)



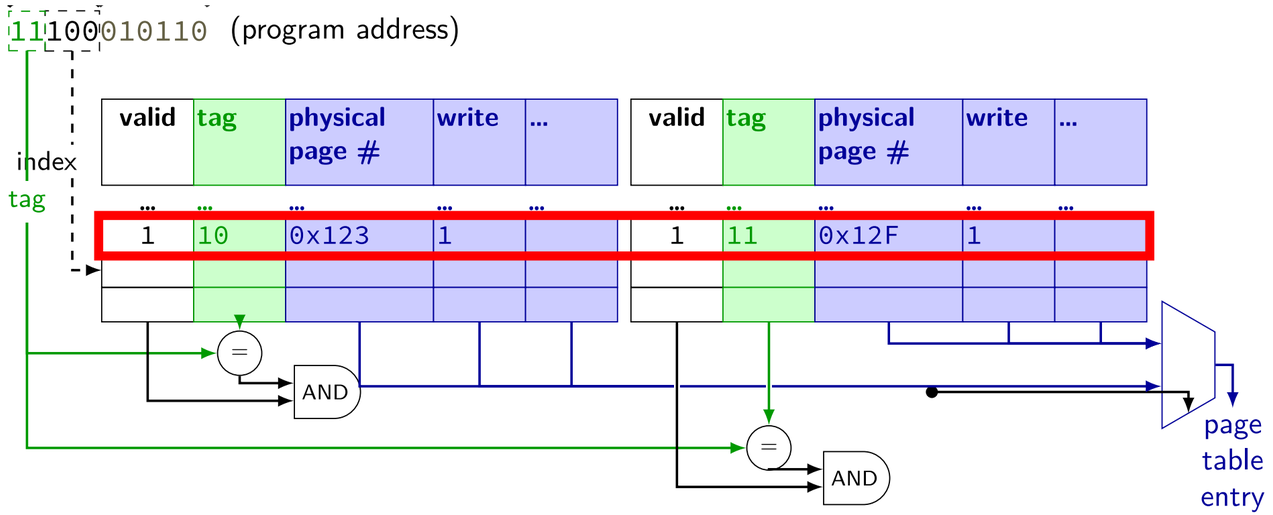
TLB organization (2-way set associative)



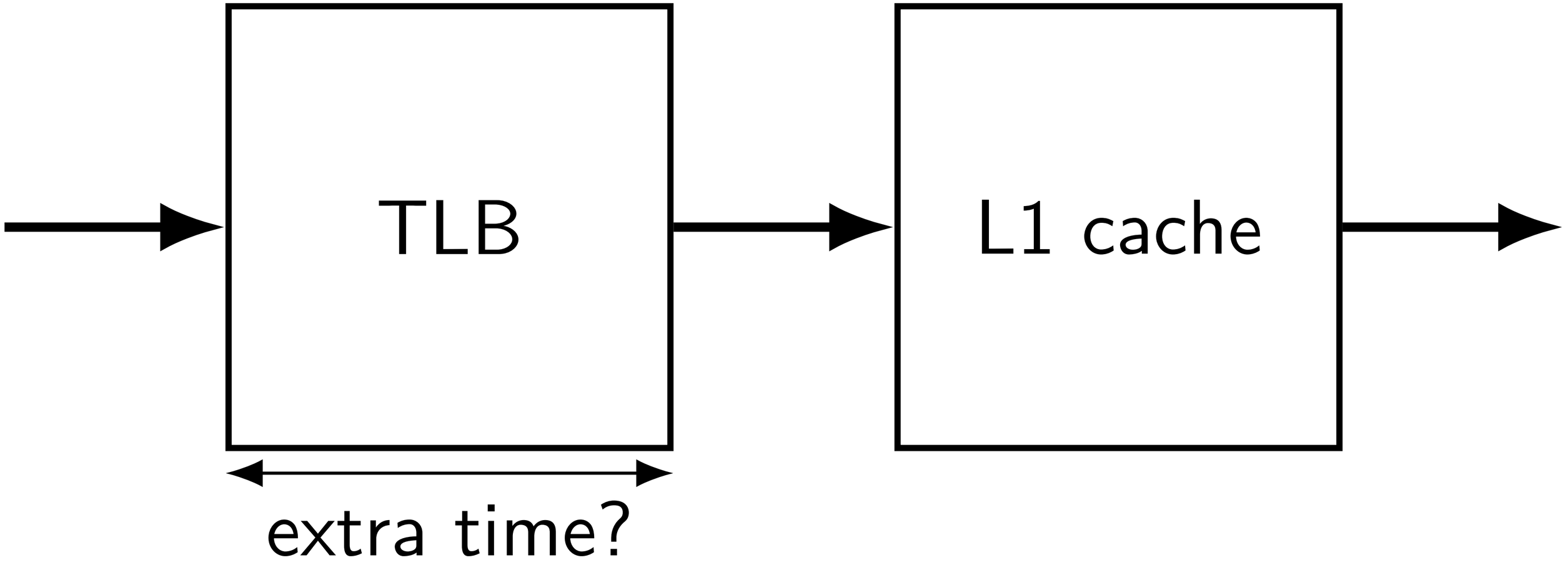
TLB organization (2-way set associative)



TLB organization (2-way set associative)



TLBs and performance

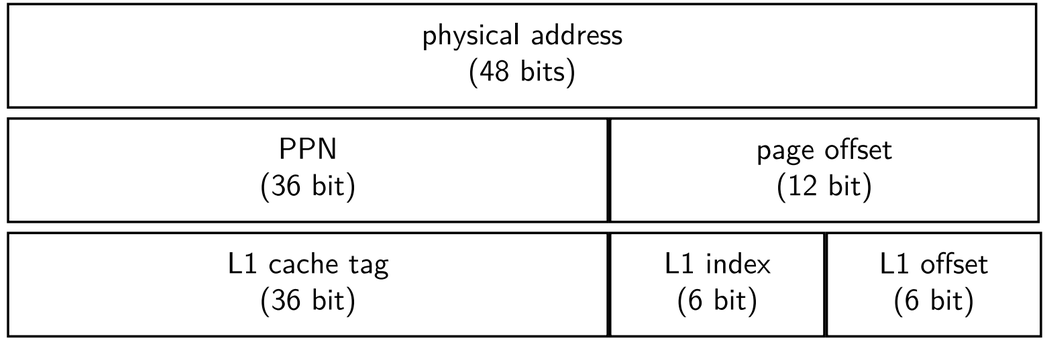


L1 caches and page numbers (Intel Skylake)

not a coincidence

why did Intel make this decision?

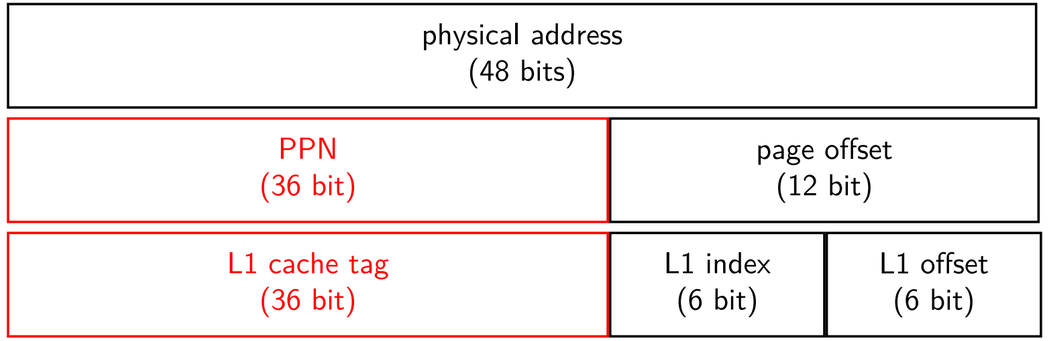
L1 caches and page numbers (Intel Skylake)



not a coincidence

why did Intel make this decision?

L1 caches and page numbers (Intel Skylake)

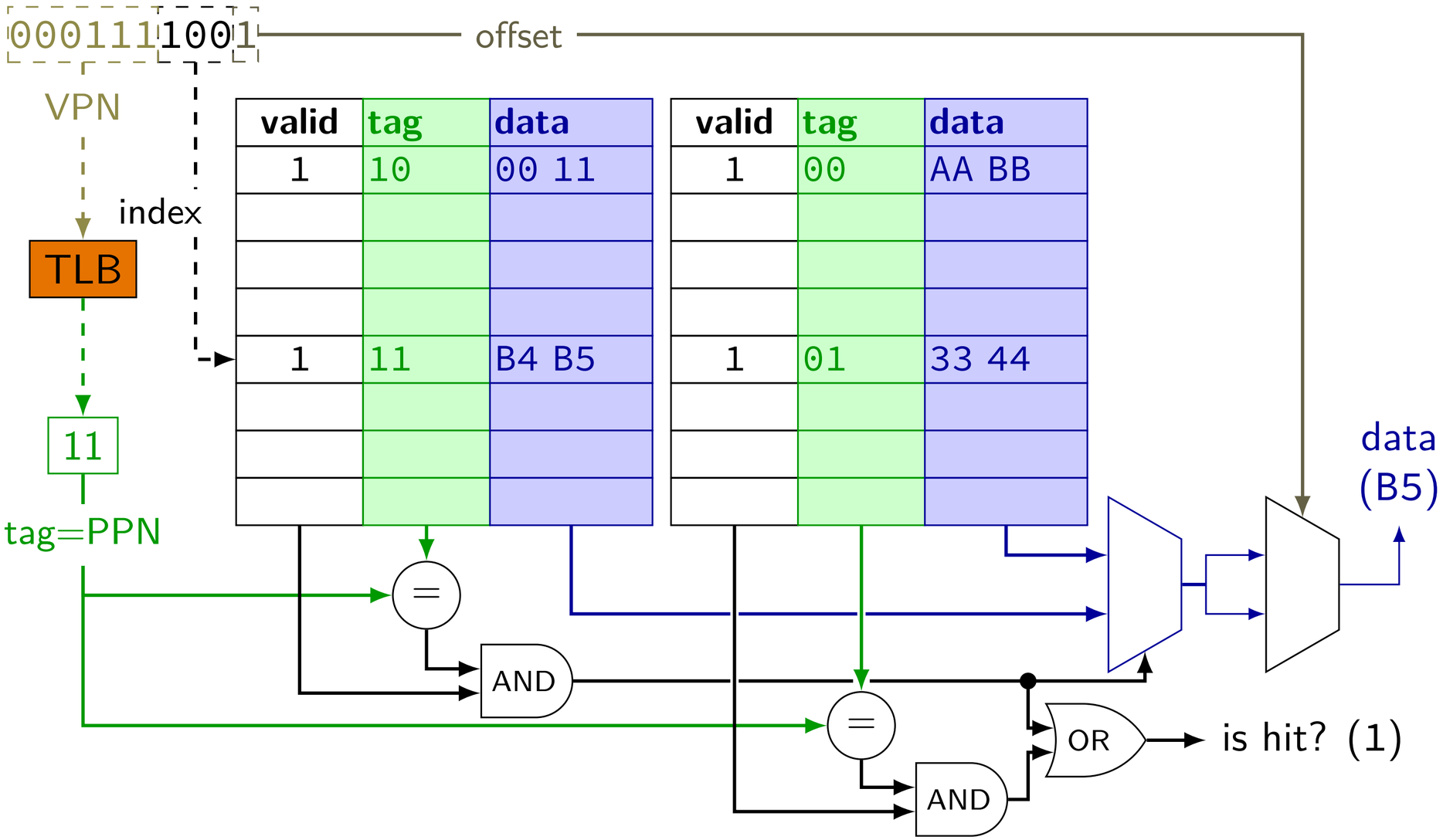


not a coincidence

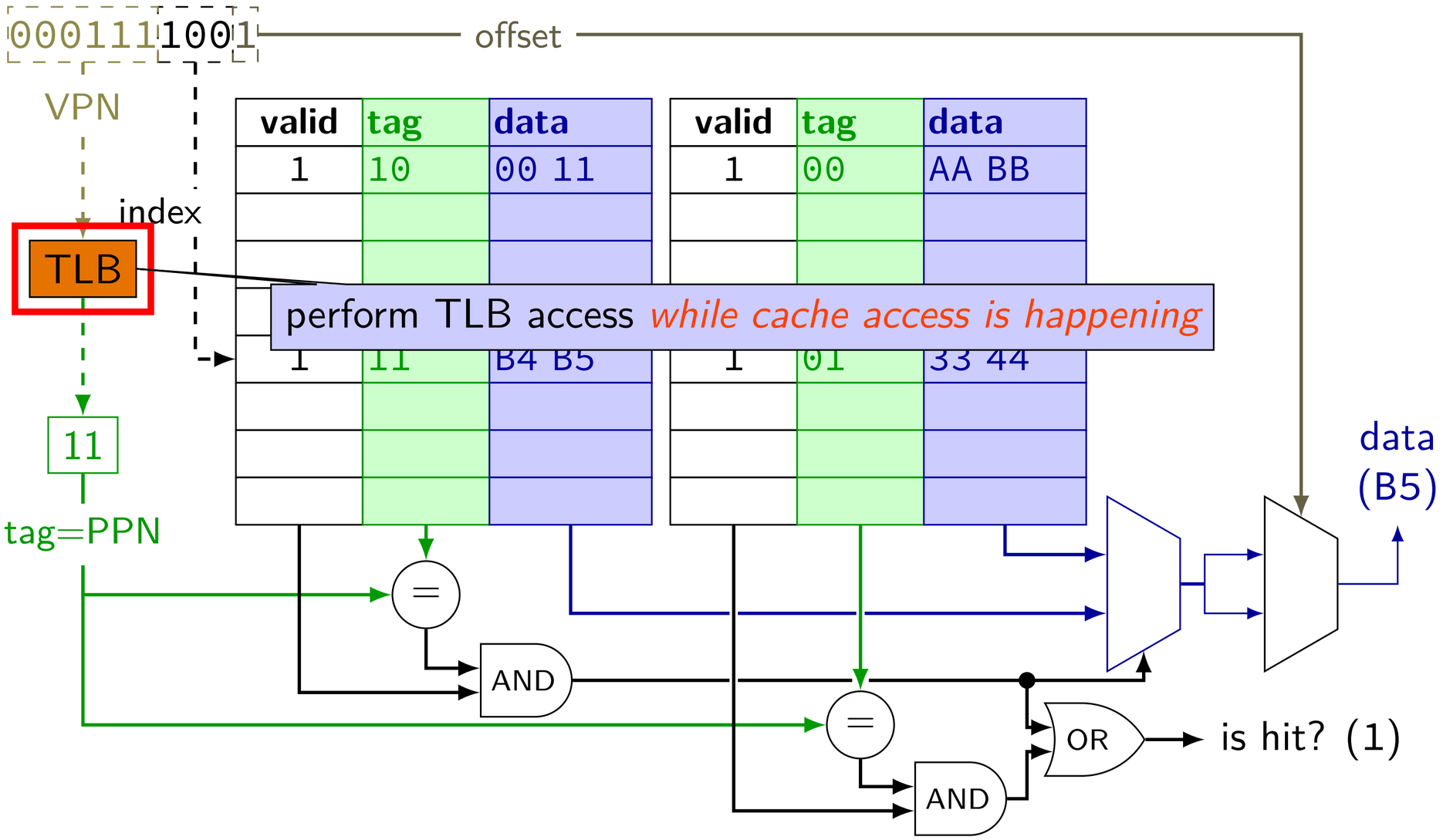
why did Intel make this decision?

overlapping TLB and cache access

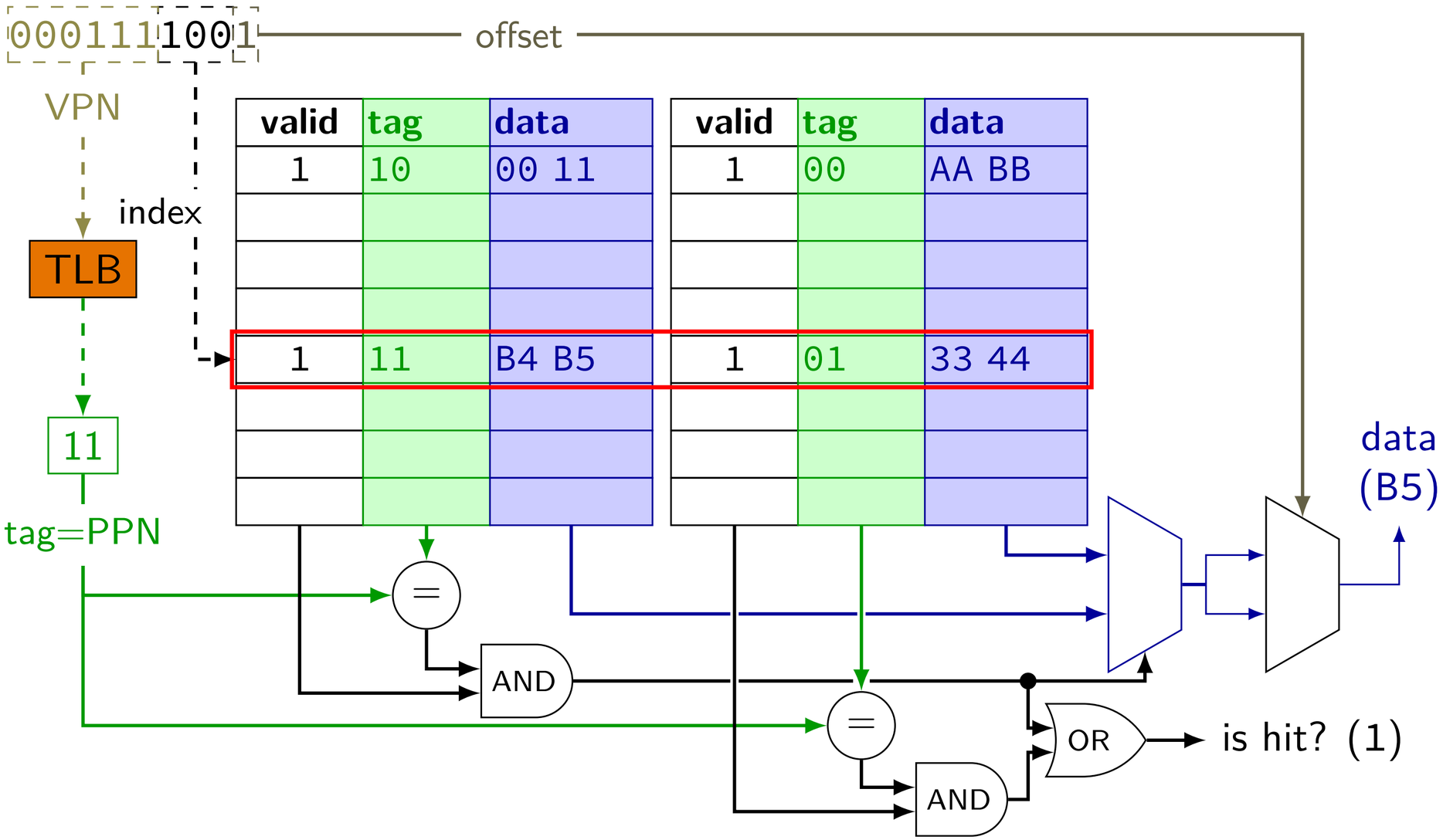
overlapping TLB and cache access



overlapping TLB and cache access



overlapping TLB and cache access



virtually-indexed, physically-tagged

called virtually-indexed, physically-tagged cache

requirement: *index contained entirely in page offset*

- do not need to do translation to start cache access

tag overlaps with PPN

- example: tag=PPN
- (but tag could include part of page offset, too)

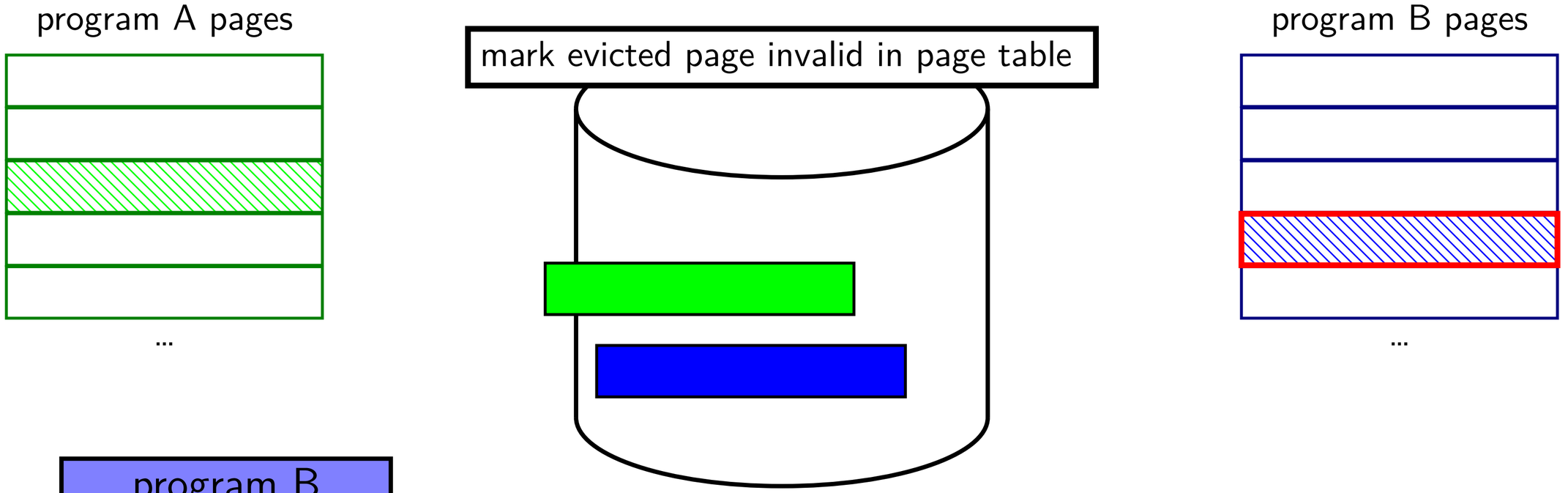
do TLB access *while retrieving cache set*

most common design in current processors

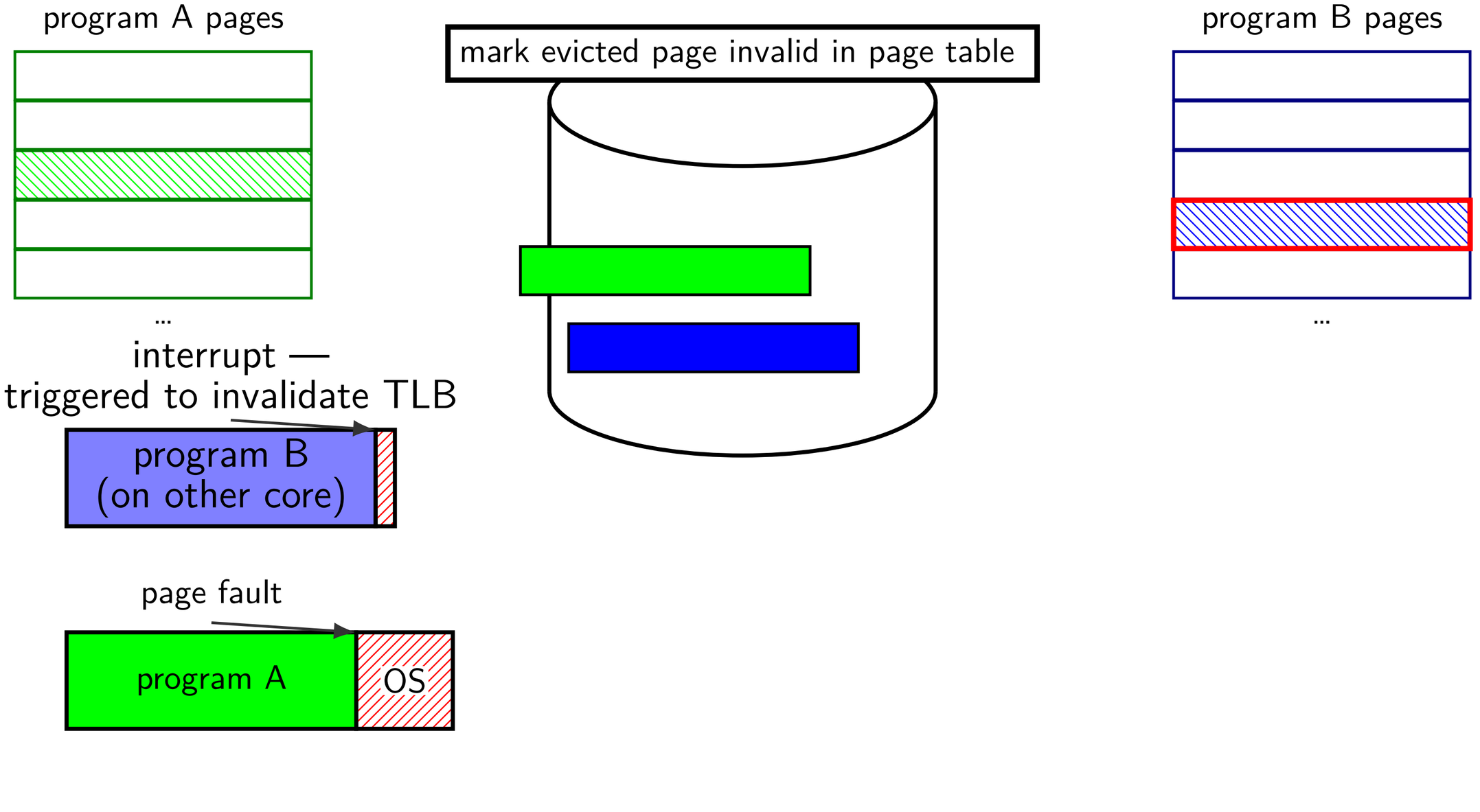
reason for highly associative (e.g. 8-way) L1 caches

TLB shootdown

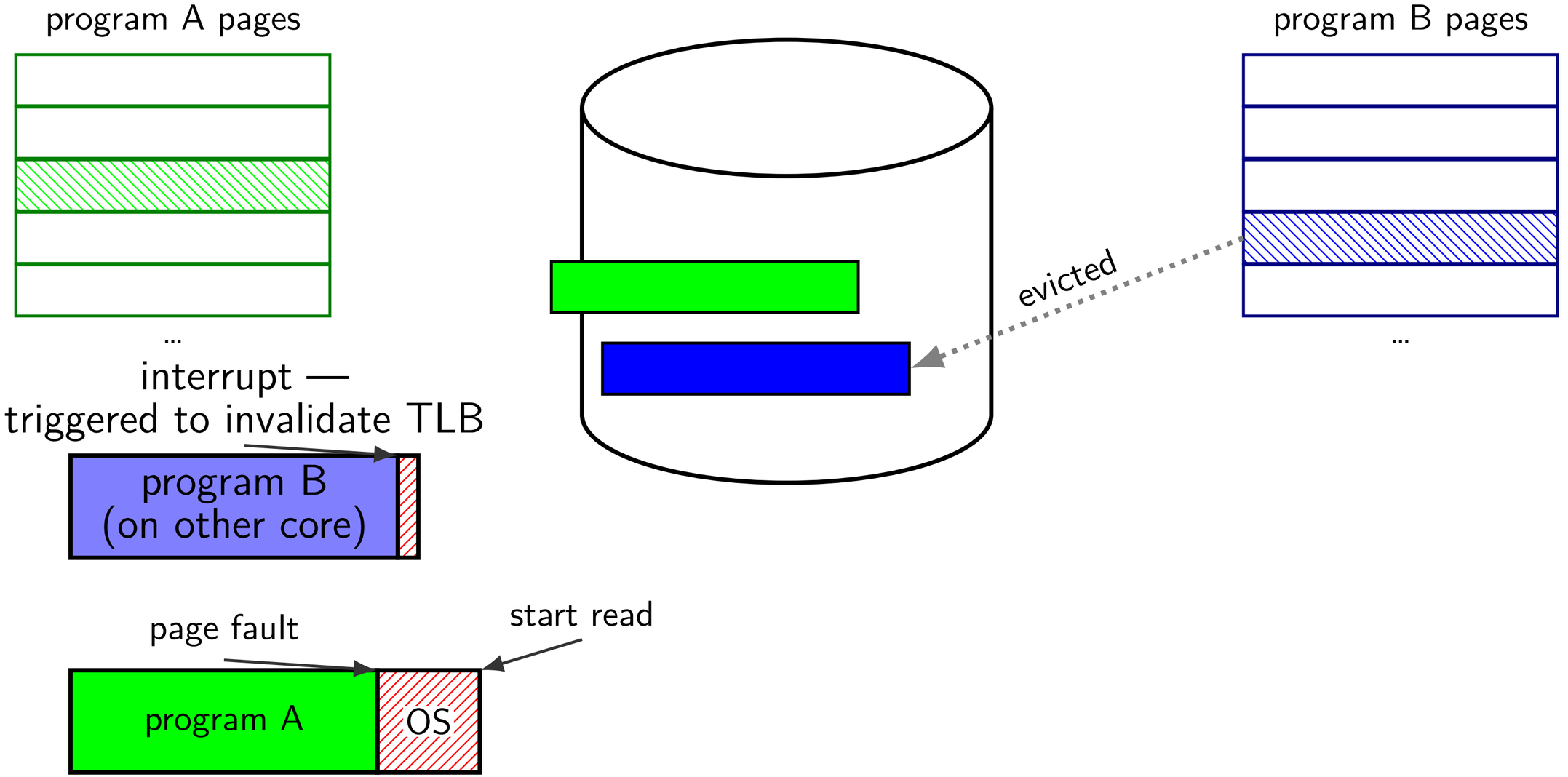
TLB shootdown



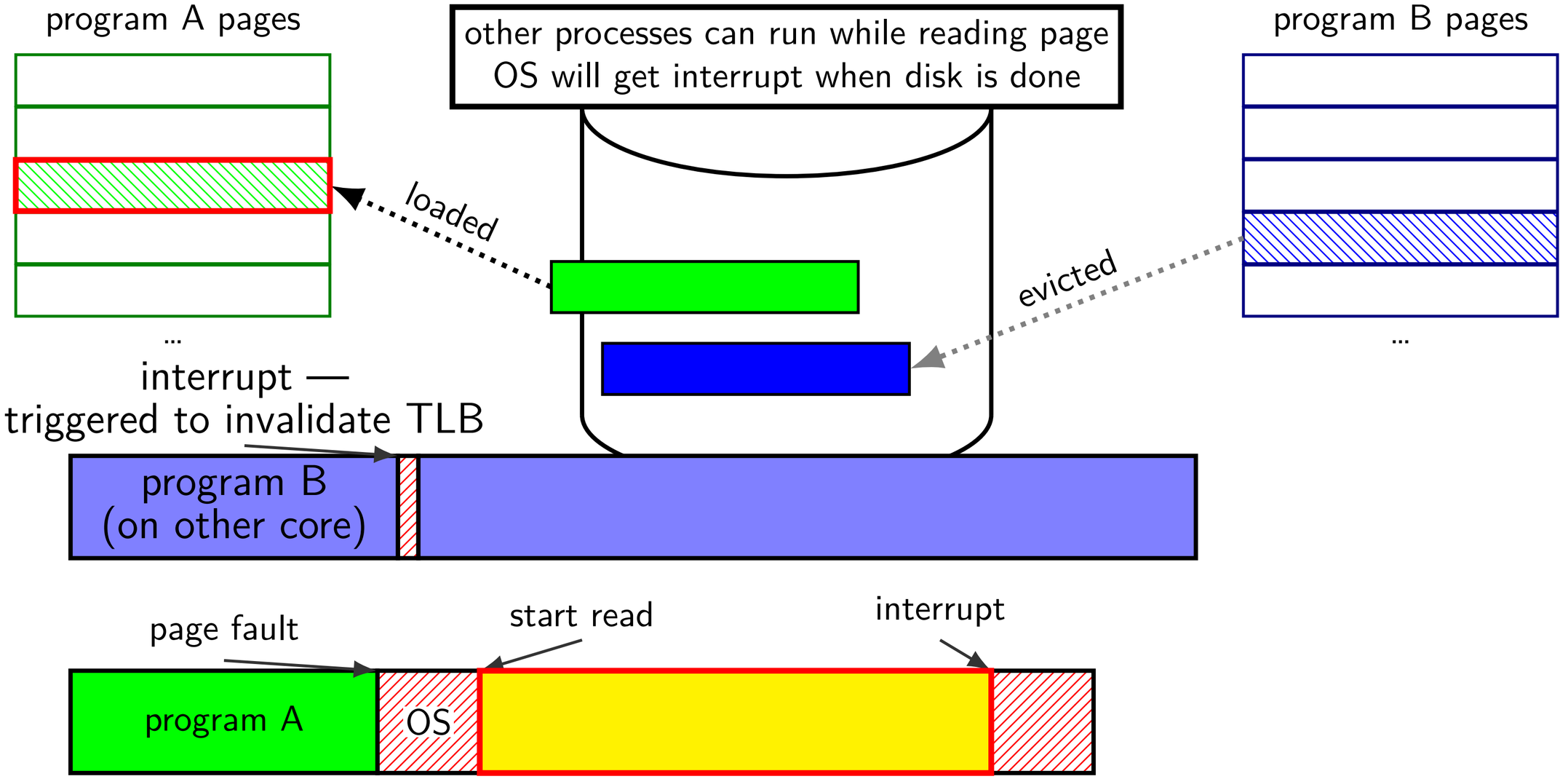
TLB shootdown



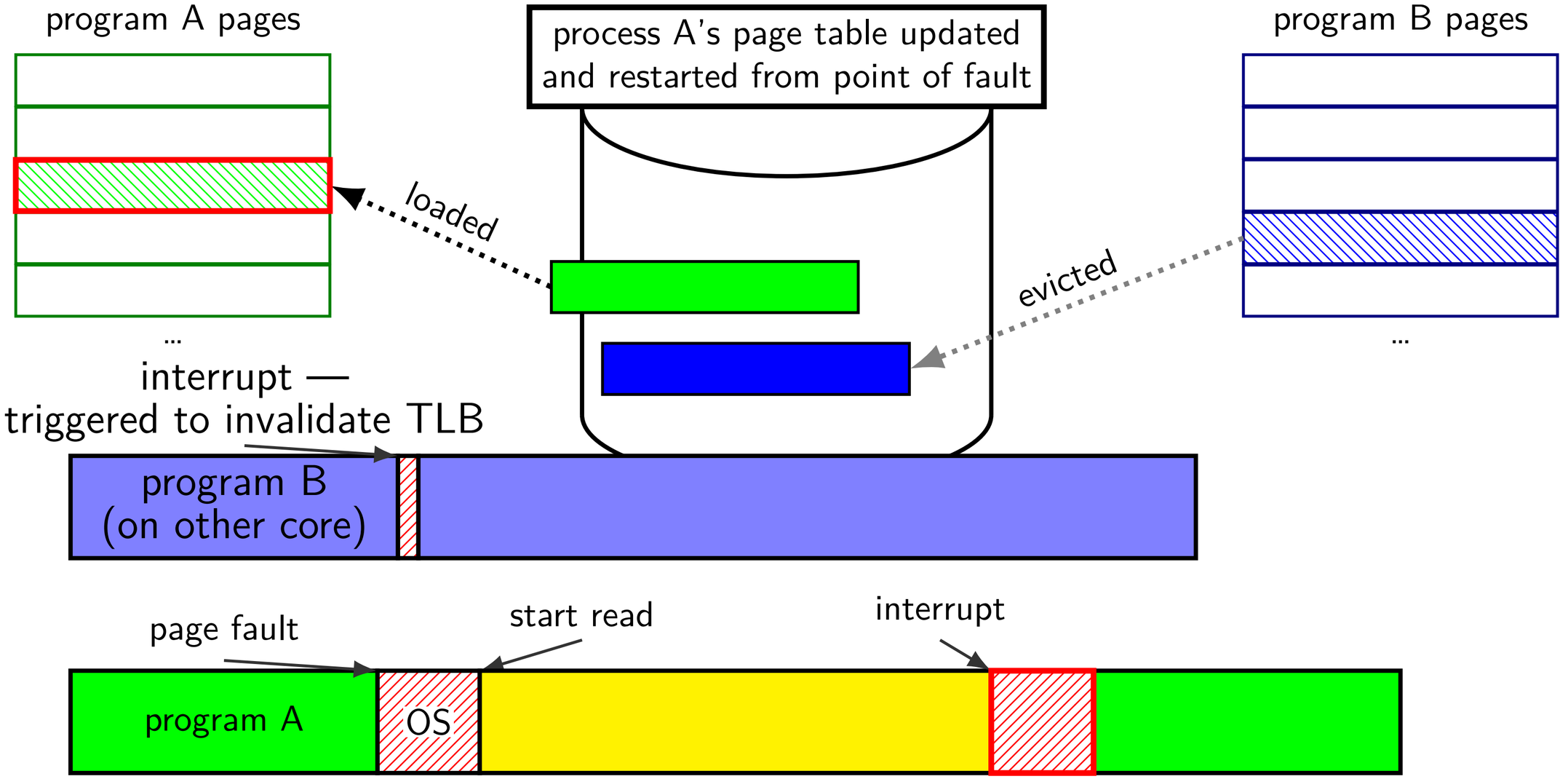
TLB shootdown



TLB shootdown



TLB shootdown



address splitting exercise (3)

384-entry, 3-way set-associative TLB

32-bit virtual address; 8KB pages

2-level page table; 4 byte PTEs

- entries in first level; in second

split the address `0x12345678`

- \iftoggle{heldback {} 13-bit page offset 1 0110 0111 1000 {} $32 - 13 = 19$-bit VPN 0001 0010 0011 0100 010 {} 8-bit first part of VPN 0001 0010 {} 11-bit second part of VPN 0011 0100 010 {} 7-bit TLB index 0100 010 {} $19 - 7 = 12$-bit TLB tag 0001 0010 0011 }

address splitting exercise (3)

384-entry, 3-way set-associative TLB

32-bit virtual address; 8KB pages

2-level page table; 4 byte PTEs

entries in first level; in second

split the address `0x12345678`

\iftoggle{heldback {} 13-bit page offset *1 0110 0111 1000* {} $32 - 13 = 19$-bit VPN 0001 0010 0011 0100 010 {} 8-bit first part of VPN 0001 0010 {} 11-bit second part of VPN 0011 0100 010 {} 7-bit TLB index 0100 010 {} $19 - 7 = 12$-bit TLB tag 0001 0010 0011 }

address splitting exercise (3)

384-entry, 3-way set-associative TLB

32-bit virtual address; 8KB pages

2-level page table; 4 byte PTEs

entries in first level; in second

split the address `0x12345678`

\iftoggle{heldback {} 13-bit page offset 1 0110 0111 1000 {} $32 - 13 = 19$-bit VPN *0001 0010 0011 0100 010* {} 8-bit first part of VPN 0001 0010 {} 11-bit second part of VPN 0011 0100 010 {} 7-bit TLB index 0100 010 {} $19 - 7 = 12$-bit TLB tag 0001 0010 0011 }

address splitting exercise (3)

384-entry, 3-way set-associative TLB

32-bit virtual address; 8KB pages

2-level page table; 4 byte PTEs

entries in first level; in second

split the address `0x12345678`

\iftoggle{heldback {} 13-bit page offset 1 0110 0111 1000 {} $32 - 13 = 19$-bit VPN 0001 0010 0011 0100 010 {} 8-bit first part of VPN *0001 0010* {} 11-bit second part of VPN 0011 0100 010 {} 7-bit TLB index 0100 010 {} $19 - 7 = 12$-bit TLB tag 0001 0010 0011 }

address splitting exercise (3)

384-entry, 3-way set-associative TLB

32-bit virtual address; 8KB pages

2-level page table; 4 byte PTEs

entries in first level; in second

split the address `0x12345678`

\iftoggle{heldback {} 13-bit page offset 1 0110 0111 1000 {} $32 - 13 = 19$-bit VPN 0001 0010 0011 0100 010 {} 8-bit first part of VPN 0001 0010 {} 11-bit second part of VPN *0011 0100 010* {} 7-bit TLB index 0100 010 {} $19 - 7 = 12$-bit TLB tag 0001 0010 0011 }

address splitting exercise (3)

384-entry, 3-way set-associative TLB

32-bit virtual address; 8KB pages

2-level page table; 4 byte PTEs

entries in first level; in second

split the address `0x12345678`

\iftoggle{heldback {} 13-bit page offset 1 0110 0111 1000 {} $32 - 13 = 19$-bit VPN 0001 0010 0011 0100 010 {} 8-bit first part of VPN 0001 0010 {} 11-bit second part of VPN 0011 0100 010 {} 7-bit TLB index *0100 010* {} $19 - 7 = 12$-bit TLB tag 0001 0010 0011 }

address splitting exercise (3)

384-entry, 3-way set-associative TLB

32-bit virtual address; 8KB pages

2-level page table; 4 byte PTEs

entries in first level; in second

split the address `0x12345678`

\iftoggle{heldback {} 13-bit page offset 1 0110 0111 1000 {} $32 - 13 = 19$-bit VPN 0001 0010 0011 0100 010 {} 8-bit first part of VPN 0001 0010 {} 11-bit second part of VPN 0011 0100 010 {} 7-bit TLB index 0100 010 {} $19 - 7 = 12$-bit TLB tag *0001 0010 0011* }